

# Catatan Ujian Akhir Semester

IF2240 Basis Data

Semester 4 Tahun 2025/2026



NARENDRA DHARMA WISTARA M.

13524044

PROGRAM STUDI TEKNIK INFORMATIKA  
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA  
INSTITUT TEKNOLOGI BANDUNG

6 Juni 2026

# Daftar Isi

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Informasi Ujian</b>                                           | <b>1</b>  |
| <b>2</b> | <b>Capaian Pembelajaran Mata Kuliah (CPMK)</b>                   | <b>1</b>  |
| 2.1      | Tabel Keterpenuhan CPMK . . . . .                                | 2         |
| <b>3</b> | <b>Daftar Materi yang Diujikan</b>                               | <b>2</b>  |
| <b>4</b> | <b>Outline Keseluruhan Materi</b>                                | <b>3</b>  |
| 4.1      | Introduction to Database . . . . .                               | 3         |
| 4.2      | Model Data . . . . .                                             | 3         |
| 4.3      | Relational Model . . . . .                                       | 3         |
| 4.4      | Relational Algebra & Calculus . . . . .                          | 3         |
| 4.5      | SQL (Structured Query Language) . . . . .                        | 3         |
| 4.6      | Database Design using E-R Model . . . . .                        | 3         |
| 4.7      | Relational Database Design . . . . .                             | 4         |
| 4.8      | Integrity Constraints . . . . .                                  | 4         |
| 4.9      | Model Data Multidimensi & Data Warehouse . . . . .               | 4         |
| <b>5</b> | <b>Introduction to Database</b>                                  | <b>5</b>  |
| 5.1      | Outline Fundamental Konsep Materi . . . . .                      | 5         |
| 5.2      | Penjelasan Konsep . . . . .                                      | 6         |
| 5.2.1    | Data, Database, dan DBMS . . . . .                               | 6         |
| 5.2.2    | Motivasi Penggunaan DBMS . . . . .                               | 6         |
| 5.2.3    | Abstraksi Data . . . . .                                         | 7         |
| 5.2.4    | Schema, Instance, dan Data Independence . . . . .                | 8         |
| 5.2.5    | Bahasa Basis Data . . . . .                                      | 8         |
| 5.2.6    | Komponen Internal DBMS . . . . .                                 | 9         |
| 5.2.7    | Arsitektur, Pengguna, dan Proses Pengembangan Database . . . . . | 10        |
| 5.3      | Algoritma dan Rumus Penting . . . . .                            | 10        |
| 5.3.1    | Alur Query Deklaratif di DBMS . . . . .                          | 10        |
| 5.3.2    | Relasi Schema dan Instance . . . . .                             | 11        |
| <b>6</b> | <b>Model Data</b>                                                | <b>12</b> |
| 6.1      | Outline Fundamental Konsep Materi . . . . .                      | 12        |
| 6.2      | Penjelasan Konsep . . . . .                                      | 12        |
| 6.2.1    | Konsep Dasar Model Data . . . . .                                | 12        |
| 6.2.2    | Level Pemodelan Data . . . . .                                   | 13        |
| 6.2.3    | Entity-Relationship Model . . . . .                              | 13        |
| 6.2.4    | Relational Model . . . . .                                       | 14        |
| 6.2.5    | Semi-Structured dan Object-Based Data Model . . . . .            | 15        |
| 6.2.6    | Network dan Hierarchical Model . . . . .                         | 15        |
| 6.2.7    | Hubungan Antar Model . . . . .                                   | 16        |
| 6.3      | Algoritma dan Rumus Penting . . . . .                            | 16        |
| 6.3.1    | Prosedur Pemodelan Data . . . . .                                | 16        |
| 6.3.2    | Transisi E-R ke Relasional . . . . .                             | 16        |
| <b>7</b> | <b>Relational Model</b>                                          | <b>18</b> |
| 7.1      | Outline Fundamental Konsep Materi . . . . .                      | 18        |
| 7.2      | Penjelasan Konsep . . . . .                                      | 18        |
| 7.2.1    | Relation, Attribute, dan Tuple . . . . .                         | 18        |
| 7.2.2    | Domain, Atomic Values, dan Null Values . . . . .                 | 19        |
| 7.2.3    | Schema dan Instance . . . . .                                    | 20        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 7.2.4     | Keys . . . . .                                                     | 20        |
| 7.2.5     | Schema Diagram dan View . . . . .                                  | 21        |
| 7.2.6     | Hubungan Relational Model dengan Materi Lain . . . . .             | 21        |
| 7.3       | Algoritma dan Rumus Penting . . . . .                              | 22        |
| 7.3.1     | Relation Schema . . . . .                                          | 22        |
| 7.3.2     | Kondisi Superkey . . . . .                                         | 22        |
| <b>8</b>  | <b>Relational Algebra &amp; Calculus</b>                           | <b>23</b> |
| 8.1       | Outline Fundamental Konsep Materi . . . . .                        | 23        |
| 8.2       | Penjelasan Konsep . . . . .                                        | 24        |
| 8.2.1     | Bahasa Kueri Relasional Formal . . . . .                           | 24        |
| 8.2.2     | Relational Algebra Dasar . . . . .                                 | 24        |
| 8.2.3     | Operator Tambahan . . . . .                                        | 25        |
| 8.2.4     | Division . . . . .                                                 | 25        |
| 8.2.5     | Extended Relational Algebra . . . . .                              | 26        |
| 8.2.6     | Relational Calculus . . . . .                                      | 26        |
| 8.2.7     | Safety, SQL Quantifier, dan Relational Completeness . . . . .      | 27        |
| 8.3       | Algoritma dan Rumus Penting . . . . .                              | 27        |
| 8.3.1     | Division . . . . .                                                 | 27        |
| 8.3.2     | Selection dan Projection . . . . .                                 | 28        |
| <b>9</b>  | <b>SQL (Structured Query Language)</b>                             | <b>29</b> |
| 9.1       | Outline Fundamental Konsep Materi . . . . .                        | 29        |
| 9.2       | Penjelasan Konsep . . . . .                                        | 30        |
| 9.2.1     | Latar Belakang dan Bagian SQL . . . . .                            | 30        |
| 9.2.2     | DDL dan Constraints . . . . .                                      | 30        |
| 9.2.3     | Query Dasar Select-From-Where . . . . .                            | 31        |
| 9.2.4     | NULL dan Three-Valued Logic . . . . .                              | 32        |
| 9.2.5     | Set Operations, Aggregation, dan Grouping . . . . .                | 32        |
| 9.2.6     | Nested Subqueries . . . . .                                        | 33        |
| 9.2.7     | Join Expressions dan Views . . . . .                               | 34        |
| 9.2.8     | Modifikasi Data, Authorization, dan Transaction . . . . .          | 34        |
| 9.2.9     | SQL Lanjutan . . . . .                                             | 35        |
| 9.3       | Algoritma dan Rumus Penting . . . . .                              | 35        |
| 9.3.1     | Urutan Logis Query SQL . . . . .                                   | 35        |
| 9.3.2     | SQL dan Aljabar Relasional . . . . .                               | 36        |
| <b>10</b> | <b>Database Design using E-R Model</b>                             | <b>37</b> |
| 10.1      | Outline Fundamental Konsep Materi . . . . .                        | 37        |
| 10.2      | Penjelasan Konsep . . . . .                                        | 38        |
| 10.2.1    | Fase Desain Basis Data . . . . .                                   | 38        |
| 10.2.2    | Entity, Attribute, dan Relationship . . . . .                      | 38        |
| 10.2.3    | Cardinality, Participation, dan Keys . . . . .                     | 39        |
| 10.2.4    | Strong Entity, Weak Entity, dan Identifying Relationship . . . . . | 39        |
| 10.2.5    | E-R Diagram dan UML Class Diagram . . . . .                        | 40        |
| 10.2.6    | Specialization, Generalization, dan Inheritance . . . . .          | 40        |
| 10.2.7    | Aggregation . . . . .                                              | 41        |
| 10.2.8    | Design Issues dalam E-R Model . . . . .                            | 41        |
| 10.2.9    | Reduksi E-R ke Skema Relasional . . . . .                          | 42        |
| 10.2.10   | Hubungan E-R Model dengan Normalisasi . . . . .                    | 43        |
| 10.3      | Algoritma dan Rumus Penting . . . . .                              | 43        |
| 10.3.1    | Key Weak Entity . . . . .                                          | 43        |
| 10.3.2    | Skema Relationship Set Many-to-Many . . . . .                      | 43        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>11 Relational Database Design</b>                                       | <b>45</b> |
| 11.1 Outline Fundamental Konsep Materi . . . . .                           | 45        |
| 11.2 Penjelasan Konsep . . . . .                                           | 46        |
| 11.2.1 Tujuan Desain Relasional . . . . .                                  | 46        |
| 11.2.2 Functional Dependency . . . . .                                     | 47        |
| 11.2.3 Closure dan Inferensi Dependensi . . . . .                          | 48        |
| 11.2.4 Canonical Cover . . . . .                                           | 49        |
| 11.2.5 Decomposition, Lossless Join, dan Dependency Preservation . . . . . | 50        |
| 11.2.6 Boyce-Codd Normal Form (BCNF) . . . . .                             | 51        |
| 11.2.7 Third Normal Form (3NF) . . . . .                                   | 52        |
| 11.2.8 Bentuk Normal Lanjutan . . . . .                                    | 53        |
| 11.2.9 Denormalization for Performance . . . . .                           | 54        |
| 11.2.10 Temporal Data Modeling . . . . .                                   | 54        |
| 11.3 Algoritma dan Rumus Penting . . . . .                                 | 55        |
| 11.3.1 Functional Dependency . . . . .                                     | 55        |
| 11.3.2 Attribute Closure . . . . .                                         | 56        |
| 11.3.3 BCNF Decomposition . . . . .                                        | 56        |
| 11.3.4 3NF Synthesis . . . . .                                             | 56        |
| <b>12 Integrity Constraints</b>                                            | <b>58</b> |
| 12.1 Outline Fundamental Konsep Materi . . . . .                           | 58        |
| 12.2 Penjelasan Konsep . . . . .                                           | 59        |
| 12.2.1 Tujuan Integrity Constraints . . . . .                              | 59        |
| 12.2.2 Pendekatan Penegakan Constraint . . . . .                           | 60        |
| 12.2.3 Domain Constraints . . . . .                                        | 60        |
| 12.2.4 Entity Integrity . . . . .                                          | 61        |
| 12.2.5 Referential Integrity . . . . .                                     | 61        |
| 12.2.6 SQL Constraints pada Relasi Tunggal . . . . .                       | 62        |
| 12.2.7 Foreign Key dan Referential Actions . . . . .                       | 63        |
| 12.2.8 Deferred Constraint Checking . . . . .                              | 63        |
| 12.2.9 Assertions, Triggers, Functions, dan Procedures . . . . .           | 64        |
| 12.2.10 Hubungan Integrity Constraints dengan Desain Relasional . . . . .  | 65        |
| 12.3 Algoritma dan Rumus Penting . . . . .                                 | 65        |
| 12.3.1 Validasi Penambahan Constraint . . . . .                            | 65        |
| 12.3.2 Kondisi Referential Integrity . . . . .                             | 66        |
| 12.3.3 Pemeriksaan Operasi DML terhadap Constraint . . . . .               | 66        |
| <b>13 Model Data Multidimensi &amp; Data Warehouse</b>                     | <b>67</b> |
| 13.1 Outline Fundamental Konsep Materi . . . . .                           | 67        |
| 13.2 Penjelasan Konsep . . . . .                                           | 68        |
| 13.2.1 Latar Belakang Decision Support . . . . .                           | 68        |
| 13.2.2 OLTP dan OLAP . . . . .                                             | 68        |
| 13.2.3 Data Warehouse . . . . .                                            | 69        |
| 13.2.4 Model Data Multidimensi . . . . .                                   | 70        |
| 13.2.5 Fact Table dan Dimension Table . . . . .                            | 70        |
| 13.2.6 Star, Snowflake, dan Galaxy Schema . . . . .                        | 71        |
| 13.2.7 Query dan Operasi OLAP . . . . .                                    | 72        |
| 13.2.8 Hubungan dengan E-R Model dan Normalisasi . . . . .                 | 72        |
| 13.3 Algoritma dan Rumus Penting . . . . .                                 | 73        |
| 13.3.1 Alur Pembangunan Data Warehouse . . . . .                           | 73        |
| 13.3.2 Bentuk Umum Query Agregasi OLAP . . . . .                           | 73        |

## 1 Informasi Ujian

|                         |                                                                                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Jenis Ujian</b>      | UAS                                                                                                                                                                               |
| <b>Nama Mata Kuliah</b> | Basis Data                                                                                                                                                                        |
| <b>Tanggal Ujian</b>    | Senin, 8 Juni 2026                                                                                                                                                                |
| <b>Sumber Utama</b>     | NotebookLM: <a href="https://notebooklm.google.com/notebook/e0b2f7c1-b643-4e02-af21-f7a6ef60a8e8">https://notebooklm.google.com/notebook/e0b2f7c1-b643-4e02-af21-f7a6ef60a8e8</a> |

Tabel 1: Informasi ujian

## 2 Capaian Pembelajaran Mata Kuliah (CPMK)

1. Menjelaskan peranan sistem basis data dalam pemenuhan kebutuhan akan informasi.
2. Menjelaskan model data relasional dan mengekspresikan kebutuhan data menggunakan operasi relasional.
3. Melakukan pemodelan data skala kecil-menengah dengan menggunakan model entity-relationship.
4. Membuat rancangan skema basis data relasional.
5. Mengimplementasikan sebuah basis data menggunakan DBMS relasional.
6. Menyusun perintah SQL untuk memperoleh data dan informasi dari basis data serta memanipulasi data di dalam basis data.

## 2.1 Tabel Keterpenuhan CPMK

| CPMK                                                                                                                   | Materi Terkait                                                                     | Keterpenuhan                                                                                     |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Menjelaskan peranan sistem basis data dalam pemenuhan kebutuhan akan informasi.                                        | Introduction to Database; Model Data Multidimensi & Data Warehouse                 | Peran DBMS, kebutuhan informasi, OLTP/OLAP, dan decision support dibahas.                        |
| Menjelaskan model data relasional dan mengekspresikan kebutuhan data menggunakan operasi relasional.                   | Relational Model; Relational Algebra & Calculus                                    | Relasi, tuple, domain, key, operator aljabar, dan kalkulus relasional dibahas.                   |
| Melakukan pemodelan data skala kecil-menengah dengan menggunakan model entity-relationship.                            | Model Data; Database Design using E-R Model                                        | Entity, relationship, atribut, cardinality, participation, weak entity, dan reduksi E-R dibahas. |
| Membuat rancangan skema basis data relasional.                                                                         | Database Design using E-R Model; Relational Database Design; Integrity Constraints | Skema relasional, FD, normal form, decomposition, dan constraints dibahas.                       |
| Mengimplementasikan sebuah basis data menggunakan DBMS relasional.                                                     | SQL; Integrity Constraints                                                         | DDL, tabel, key, foreign key, check, view, trigger, dan transaction dibahas.                     |
| Menyusun perintah SQL untuk memperoleh data dan informasi dari basis data serta memanipulasi data di dalam basis data. | SQL; Model Data Multidimensi & Data Warehouse                                      | Query dasar, join, agregasi, subquery, insert, delete, update, dan query analitik dibahas.       |

Tabel 2: Keterpenuhan CPMK per materi yang dibahas

## 3 Daftar Materi yang Diujikan

1. Introduction to Database
2. Model Data
3. Relational Model
4. Relational Algebra & Calculus
5. SQL (Structured Query Language)
6. Database Design using E-R Model
7. Relational Database Design

- 8. Integrity Constraints
- 9. Model Data Multidimensi & Data Warehouse

## 4 Outline Keseluruhan Materi

Bagian ini menyajikan peta belajar ringkas untuk seluruh materi yang diujikan. Penjelasan mendalam tersedia pada section masing-masing materi.

### 4.1 Introduction to Database

- Data, basis data, DBMS, peran DBMS, abstraksi data, schema-instance, data independence, komponen DBMS, dan bahasa basis data.
- Proses pengembangan basis data melalui data modeling, SDLC, dan prototyping/RAD.

### 4.2 Model Data

- Tujuan pemodelan, level conceptual-logical-physical, dan prosedur identifikasi grup data.
- Kategori model data: E-R, relational, semi-structured, object-based, network, hierarchical, dan hubungan antar model.

### 4.3 Relational Model

- Relation, attribute, tuple, domain, atomic value, schema, instance, key, foreign key, null, dan integrity constraints.
- Relational model menjadi dasar SQL, aljabar relasional, kalkulus relasional, dan normalisasi.

### 4.4 Relational Algebra & Calculus

- Operator dasar aljabar relasional: selection, projection, union, set difference, Cartesian product, dan rename.
- Operator turunan, aggregation, null, multiset semantics, tuple relational calculus, domain relational calculus, dan relational completeness.

### 4.5 SQL (Structured Query Language)

- DDL, DML, query dasar, null logic, set operations, aggregation, grouping, joins, nested subquery, views, modification, authorization, dan transaction.
- Advanced SQL mencakup stored procedure, trigger, JDBC/ODBC, prepared statement, recursive query, window function, cube, dan rollup.

### 4.6 Database Design using E-R Model

- Fase desain basis data, entity, attribute, relationship, key, cardinality, participation, weak entity, dan E-R diagram.
- Extended E-R, UML, isu desain, dan reduksi E-R ke skema relasional.

#### **4.7 Relational Database Design**

- Tujuan desain relasional, universal relation, anomaly, functional dependency, closure, key, canonical cover, decomposition, lossless join, dan dependency preservation.
- Normal form: 1NF, 2NF, 3NF, BCNF, 4NF, serta trade-off normalisasi, denormalization, dan temporal data modeling.

#### **4.8 Integrity Constraints**

- Domain, entity, referential integrity, SQL constraints, named constraints, foreign key action, deferred checking, assertions, triggers, functions, dan procedures.

#### **4.9 Model Data Multidimensi & Data Warehouse**

- OLTP vs OLAP, data warehouse, ETL/ELT, fact, measure, dimension, member, data cube, grain, star schema, snowflake schema, galaxy schema, dan OLAP operations.



## 5 Introduction to Database

Introduction to Database menjelaskan mengapa sistem basis data dibutuhkan, bagaimana DBMS menyembunyikan kompleksitas penyimpanan data, dan komponen apa saja yang membuat data dapat disimpan, diambil, diamankan, dan dimodifikasi secara konsisten. Materi ini adalah fondasi untuk semua materi berikutnya: model data, model relasional, SQL, desain E-R, normalisasi, integrity constraints, dan data warehouse.

Fokus utama untuk ujian adalah memahami perbedaan data, database, dan DBMS; alasan DBMS lebih baik daripada file-processing system; level abstraksi data; schema-instance; data independence; bahasa basis data; serta komponen internal DBMS.

### 5.1 Outline Fundamental Konsep Materi

1. **WAJIB** Konsep dasar data dan database
  - (a) **WAJIB** Data
  - (b) **WAJIB** Data terstruktur dan tidak terstruktur
  - (c) **WAJIB** Basis data
  - (d) **WAJIB** DBMS
2. **WAJIB** Motivasi penggunaan DBMS
  - (a) **WAJIB** Masalah file-processing system
  - (b) **WAJIB** Peran DBMS dalam penyimpanan, akses, keamanan, recovery, dan konkurensi
3. **WAJIB** Abstraksi data
  - (a) **WAJIB** Physical level
  - (b) **WAJIB** Logical level
  - (c) **WAJIB** View level
4. **WAJIB** Schema, instance, dan data independence
  - (a) **WAJIB** Physical schema, logical schema, dan subschema/view
  - (b) **WAJIB** Instance
  - (c) **WAJIB** Physical data independence
  - (d) **PAHAM** Logical data independence
5. **WAJIB** Bahasa basis data
  - (a) **WAJIB** Data Definition Language (DDL)
  - (b) **WAJIB** Data Manipulation Language (DML)
  - (c) **WAJIB** Procedural dan declarative query language
  - (d) **PAHAM** Authorization dan transaction control
6. **WAJIB** Komponen internal DBMS
  - (a) **WAJIB** Storage manager
  - (b) **WAJIB** Query processor
  - (c) **WAJIB** Transaction manager
7. **PAHAM** Arsitektur, pengguna, dan pengembangan database
  - (a) **PAHAM** Centralized, client-server, parallel, dan distributed architecture
  - (b) **PAHAM** Database users
  - (c) **PAHAM** Data modeling, SDLC, dan prototyping/RAD

## 5.2 Penjelasan Konsep

### 5.2.1 Data, Database, dan DBMS

#### Inti Konsep.

Data adalah representasi tersimpan dari objek dan peristiwa bermakna; database adalah kumpulan data yang terorganisir dan saling berhubungan secara logis; DBMS adalah sistem perangkat lunak untuk menyimpan, mengelola, dan mengakses database. Ketiga konsep ini penting karena seluruh mata kuliah Basis Data berangkat dari kebutuhan mengubah data mentah menjadi informasi yang dapat dipakai secara efisien dan aman.

Motivasi konsep ini adalah organisasi perlu menyimpan fakta tentang dunia nyata: mahasiswa, dosen, mata kuliah, departemen, transaksi, produk, dan sebagainya. Fakta tersebut tidak cukup hanya diletakkan sebagai file acak; ia perlu diorganisasi agar dapat dicari, diperbarui, diamankan, dan dibagikan ke banyak pengguna.

**Data** adalah representasi tersimpan dari objek dan peristiwa yang memiliki makna. Sumber membedakan data menjadi **data terstruktur**, seperti angka, teks, dan tanggal, serta **data tidak terstruktur**, seperti gambar, video, dan dokumen.

**Database** adalah sekumpulan data yang terorganisir dan saling berhubungan secara logis. Keterhubungan logis berarti data tidak berdiri sendiri: data mahasiswa berhubungan dengan pengambilan mata kuliah, data instructor berhubungan dengan department, dan data course berhubungan dengan section.

**Database Management System (DBMS)** adalah kumpulan data yang saling berelasi beserta program untuk mengakses dan mengelola data tersebut. DBMS bertugas menyediakan cara yang nyaman dan efisien untuk menyimpan serta mengambil informasi.

| Konsep   | Makna dan Contoh                                                                                                                     |
|----------|--------------------------------------------------------------------------------------------------------------------------------------|
| Data     | Representasi objek/peristiwa bermakna. Contoh: 13524044, Basis Data, 8 Juni 2026.                                                    |
| Database | Kumpulan data terorganisir. Contoh: database universitas berisi student, instructor, course, dan department.                         |
| DBMS     | Sistem untuk menyimpan, mengambil, mengamankan, dan mengelola database. Contoh fungsi: query, update, recovery, concurrency control. |

Tabel 3: Perbedaan data, database, dan DBMS

Konsep ini menjadi prasyarat model data. Setelah memahami bahwa database menyimpan data yang terhubung, kita membutuhkan model data untuk mendeskripsikan struktur dan hubungan data tersebut.

### 5.2.2 Motivasi Penggunaan DBMS

#### Inti Konsep.

DBMS digunakan karena file-processing system tradisional sulit menjaga konsistensi, efisiensi, keamanan, recovery, dan akses bersama. DBMS menyediakan mekanisme terpusat untuk mengelola data besar secara aman dan konsisten.

Sebelum DBMS, data sering disimpan dalam file terpisah yang dikelola masing-masing program aplikasi. Pendekatan ini disebut **file-processing system**. Masalahnya, setiap aplikasi dapat memiliki format file sendiri, aturan validasi sendiri, dan salinan data sendiri.

Masalah utama file-processing system:

1. **Redundansi data:** fakta yang sama disimpan di banyak file.

2. **Inkonsistensi data:** salinan data yang sama dapat memiliki nilai berbeda.
3. **Kesulitan akses:** query baru sering membutuhkan program baru.
4. **Isolasi data:** data tersebar dalam banyak file dan format.
5. **Masalah integritas:** aturan data harus diprogram manual di banyak aplikasi.
6. **Masalah atomicity:** operasi yang gagal di tengah dapat meninggalkan data setengah berubah.
7. **Masalah concurrent access:** banyak pengguna dapat mengubah data bersamaan dan menghasilkan anomali.
8. **Masalah keamanan:** sulit mengatur hak akses granular.

DBMS mengatasi masalah tersebut dengan menyimpan data dalam sistem terpadu, menyediakan bahasa query, menegakkan constraint, mengatur transaksi, dan mengelola akses bersamaan.

| Peran DBMS                      | Penjelasan                                                               |
|---------------------------------|--------------------------------------------------------------------------|
| Efisiensi penyimpanan dan akses | DBMS mengatur struktur penyimpanan, indeks, buffer, dan query execution. |
| Keamanan                        | DBMS membatasi pengguna berdasarkan authorization.                       |
| Recovery                        | DBMS menjaga database tetap konsisten meskipun terjadi crash.            |
| Concurrency control             | DBMS mengatur akses banyak pengguna agar transaksi tidak saling merusak. |
| Integrity enforcement           | DBMS menolak data yang melanggar constraint.                             |

Tabel 4: Peran utama DBMS

Materi ini berhubungan dengan transaction management dan integrity constraints. Tanpa DBMS, atomicity, consistency, dan concurrency harus ditangani sendiri oleh aplikasi, yang rawan salah dan sulit diskalakan.

### 5.2.3 Abstraksi Data

#### Inti Konsep.

Abstraksi data adalah cara DBMS menyembunyikan detail penyimpanan fisik agar pengguna dapat bekerja pada struktur logis atau tampilan yang relevan. Konsep ini penting karena pengguna tidak perlu memahami byte, blok disk, indeks, atau file layout untuk menulis query.

Motivasi abstraksi data adalah kompleksitas. Data sebenarnya tersimpan sebagai struktur rendah di media penyimpanan, tetapi pengguna biasanya berpikir dalam bentuk tabel, atribut, dan hubungan. DBMS memisahkan cara data disimpan dari cara data dipahami.

| Level                 | Definisi dan Contoh                                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Physical level</b> | Level terendah; mendeskripsikan bagaimana data disimpan secara fisik, seperti blok disk, alokasi file, dan struktur indeks. |
| <b>Logical level</b>  | Mendeskripsikan struktur logis keseluruhan database: tabel, atribut, tipe data, dan hubungan antar data.                    |
| <b>View level</b>     | Level tertinggi; menampilkan sebagian database yang relevan untuk pengguna atau aplikasi tertentu.                          |

Tabel 5: Tiga level abstraksi data

Mekanisme kerjanya:

1. Storage manager menangani detail fisik.
2. Logical schema mendefinisikan struktur data yang dipakai query.
3. View menyajikan subset atau transformasi data untuk kebutuhan tertentu.
4. Pengguna berinteraksi dengan view atau schema logis tanpa perlu mengetahui detail fisik.

Abstraksi data menjadi fondasi view di SQL dan data independence. Ketika pengguna menulis query pada tabel atau view, DBMS menerjemahkan query tersebut ke operasi fisik yang sesuai.

#### 5.2.4 Schema, Instance, dan Data Independence

##### Inti Konsep.

Schema adalah rancangan struktur database, sedangkan instance adalah isi aktual database pada satu titik waktu. Data independence adalah kemampuan mengubah satu level rancangan tanpa memaksa perubahan besar pada level lain.

Motivasi membedakan schema dan instance adalah struktur database relatif stabil, sementara isinya berubah terus-menerus. Tabel `student(ID, name, dept_name, tot_cred)` adalah schema; kumpulan mahasiswa aktual pada hari ini adalah instance.

| Istilah         | Definisi                                                                      |
|-----------------|-------------------------------------------------------------------------------|
| Physical schema | Deskripsi struktur fisik penyimpanan data.                                    |
| Logical schema  | Deskripsi struktur logis database secara keseluruhan.                         |
| Subschema/View  | Deskripsi bagian database yang terlihat oleh pengguna atau aplikasi tertentu. |
| Instance        | Isi aktual database pada satu titik waktu.                                    |

Tabel 6: Schema dan instance

Analogi sumber: schema seperti deklarasi tipe data dalam bahasa pemrograman, sedangkan instance seperti nilai variabel pada saat eksekusi. Tipe variabel relatif tetap; nilai variabel berubah-ubah.

**Physical data independence** adalah kemampuan mengubah physical schema tanpa mengubah logical schema atau program aplikasi. Misalnya, menambah indeks untuk mempercepat query tidak seharusnya mengubah query aplikasi.

**Logical data independence** adalah kemampuan mengubah logical schema tanpa mengubah view atau aplikasi. Ini lebih sulit dicapai karena perubahan struktur logis, seperti memecah tabel atau mengganti atribut, lebih dekat dengan cara aplikasi memahami data.

Data independence berhubungan dengan desain skema dan SQL view. View dapat melindungi pengguna dari sebagian perubahan logis, sedangkan storage manager melindungi pengguna dari perubahan fisik.

#### 5.2.5 Bahasa Basis Data

##### Inti Konsep.

Bahasa basis data memungkinkan pengguna mendefinisikan struktur database dan memanipulasi datanya. DDL mendefinisikan schema, sedangkan DML mengambil, menyisipkan, menghapus, dan memperbarui data.

Motivasi bahasa basis data adalah pengguna membutuhkan cara formal untuk berkomunikasi dengan DBMS. Tanpa bahasa standar, setiap operasi data harus ditulis langsung sebagai program rendah yang mengakses file.

| Bahasa/Fitur        | Peran                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------|
| DDL                 | Mendefinisikan schema, domain, constraint, view, dan metadata. Hasil kompilasi DDL disimpan dalam data dictionary. |
| DML                 | Mengambil, menyisipkan, menghapus, dan memperbaiki data.                                                           |
| Procedural DML      | Pengguna menyebut data yang dicari dan cara mendapatkannya.                                                        |
| Declarative DML     | Pengguna menyebut data yang diinginkan tanpa menentukan algoritma pencarian. SQL terutama bersifat deklaratif.     |
| Authorization       | Mengatur hak akses seperti read, insert, update, dan delete.                                                       |
| Transaction control | Menentukan awal, akhir, commit, dan rollback transaksi.                                                            |

Tabel 7: Bahasa dan fitur basis data

DDL berhubungan erat dengan integrity constraints karena constraint seperti domain, primary key, dan referential integrity didefinisikan pada schema. DML berhubungan erat dengan SQL dan relational algebra karena query harus diterjemahkan ke operasi yang dapat dievaluasi DBMS.

### 5.2.6 Komponen Internal DBMS

#### Inti Konsep.

DBMS terdiri dari subsistem yang mengelola penyimpanan, pemrosesan query, dan transaksi. Komponen ini penting karena DBMS tidak hanya menyimpan data, tetapi juga menerjemahkan query, mengoptimasi eksekusi, menjaga keamanan, dan memulihkan database dari kegagalan.

Motivasi pembagian komponen adalah kompleksitas tugas DBMS. Menjalankan query SQL sederhana dapat melibatkan parsing, optimasi, pemilihan indeks, pembacaan disk, penggunaan buffer, pengecekan authorization, dan pencatatan transaksi.

| Komponen            | Tugas Utama                                                                                                                                                                       |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Storage manager     | Menjadi antarmuka antara data fisik di disk dan query/program aplikasi. Mencakup file manager, buffer manager, authorization and integrity manager, data dictionary, dan indices. |
| Query processor     | Memproses DDL dan DML. Mencakup DDL interpreter, DML compiler, query optimization, dan query evaluation engine.                                                                   |
| Transaction manager | Menjaga database tetap konsisten meskipun ada crash dan akses bersamaan. Mencakup recovery dan concurrency-control manager.                                                       |

Tabel 8: Komponen internal DBMS

Mekanisme query secara ringkas:

1. Pengguna mengirim query.
2. Query processor menerjemahkan dan mengoptimasi query.
3. Storage manager mengambil data melalui file, indeks, dan buffer.
4. Authorization dan integrity manager memeriksa hak akses dan constraint.
5. Transaction manager memastikan eksekusi aman terhadap crash dan concurrency.

Komponen ini menjelaskan mengapa SQL yang deklaratif bisa dijalankan tanpa pengguna menentukan algoritma. Pengguna menyatakan apa yang diinginkan; DBMS menentukan cara eksekusi fisiknya.

### 5.2.7 Arsitektur, Pengguna, dan Proses Pengembangan Database

#### Inti Konsep.

Database dapat dibangun dalam berbagai arsitektur dan digunakan oleh berbagai jenis pengguna. Pemodelan data dan proses pengembangan seperti SDLC atau prototyping menentukan bagaimana kebutuhan organisasi diterjemahkan menjadi database yang dapat digunakan.

Arsitektur database menentukan lokasi penyimpanan dan pemrosesan. **Centralized architecture** menempatkan sistem pada satu mesin. **Client-server architecture** memisahkan client dan database server; pada two-tier, client berkomunikasi langsung dengan database server, sedangkan pada three-tier, application server menjadi perantara. **Parallel architecture** memakai banyak CPU/perangkat keras untuk memproses data besar lebih cepat. **Distributed architecture** menyebarkan data dan pemrosesan ke banyak mesin, bahkan lokasi geografis berbeda.

| Pengguna                     | Cara Berinteraksi                                                                   |
|------------------------------|-------------------------------------------------------------------------------------|
| Naive/operational user       | Menggunakan aplikasi siap pakai, misalnya clerk yang memproses transaksi.           |
| Analyst/knowledge worker     | Menjalankan query dan analisis untuk decision support.                              |
| Application programmer       | Menulis program yang mengakses database.                                            |
| Database Administrator (DBA) | Mengelola schema, authorization, tuning, backup, recovery, dan administrasi sistem. |

Tabel 9: Jenis pengguna database

**Data modeling** adalah teknik untuk mengoptimalkan cara informasi disimpan dan digunakan organisasi. Mekanismenya dimulai dengan mengidentifikasi grup data utama, lalu mendefinisikan konten rinci dari tiap grup. E-R model menjadi alat konseptual utama untuk tahap ini.

**System Development Life Cycle (SDLC)** adalah proses pengembangan yang detail dan terencana. Ia menghasilkan rancangan komprehensif, tetapi membutuhkan waktu panjang. **Prototyping/RAD** lebih cepat: pemodelan awal dilakukan secara kasar, database dibentuk pada prototype awal, lalu diperbaiki berulang berdasarkan implementasi dan feedback.

Bagian ini berhubungan langsung dengan E-R model dan relational database design. Kebutuhan pengguna diterjemahkan ke model konseptual, direduksi ke skema relasional, lalu dinormalisasi agar bebas anomali.

## 5.3 Algoritma dan Rumus Penting

### 5.3.1 Alur Query Deklaratif di DBMS

#### Tujuan Algoritma.

Pseudocode ini merangkum bagaimana DBMS mengeksekusi query deklaratif: pengguna menyatakan hasil yang diinginkan, DBMS menentukan rencana fisiknya.

Listing 1: Pseudocode alur pemrosesan query

```

1 Input: declarative query Q
2 Output: query result
3

```

```
4 parse Q and validate syntax
5 check authorization and relevant integrity constraints
6 translate Q into logical query plan
7 optimize logical plan into physical execution plan
8 execute plan using storage manager, indexes, and buffers
9 return result to user
```

### 5.3.2 Relasi Schema dan Instance

#### Makna Rumus.

Notasi ini membedakan struktur relasi dari isi aktualnya. Struktur relatif stabil, sedangkan isi relasi berubah dari waktu ke waktu.

$$R(A_1, A_2, \dots, A_n) \tag{1}$$

dengan:

- $R$ : nama skema relasi.
- $A_1, A_2, \dots, A_n$ : atribut pada relasi.
- $r(R)$ : instance aktual yang berisi tuple-tuple pada satu titik waktu.

## 6 Model Data

Model Data membahas cara mendeskripsikan data, hubungan antar data, makna data, dan batasan konsistensi sebelum database benar-benar diimplementasikan. Materi ini penting karena desain database tidak dimulai dari SQL, tetapi dari pemahaman konseptual tentang informasi apa yang perlu disimpan dan bagaimana informasi itu saling berhubungan.

Fokus utama untuk ujian adalah memahami tujuan pemodelan data, level conceptual-logical-physical, jenis-jenis model data, serta bagaimana E-R model menjadi rancangan konseptual yang kemudian diterjemahkan ke model relasional.

### 6.1 Outline Fundamental Konsep Materi

1. **WAJIB** Konsep dasar model data
  - (a) **WAJIB** Definisi model data
  - (b) **WAJIB** Tujuan pemodelan data
  - (c) **WAJIB** Prosedur identifikasi grup data dan konten detail
2. **WAJIB** Level pemodelan data
  - (a) **WAJIB** Conceptual data model
  - (b) **WAJIB** Logical data model
  - (c) **WAJIB** Physical data model
3. **WAJIB** Kategori model data
  - (a) **WAJIB** Entity-Relationship Model
  - (b) **WAJIB** Relational Model
  - (c) **PAHAM** Semi-structured Data Model
  - (d) **PAHAM** Object-Based Data Model
  - (e) **PAHAM** Network Model
  - (f) **PAHAM** Hierarchical Model
4. **WAJIB** Hubungan antar model
  - (a) **WAJIB** E-R model sebagai desain konseptual
  - (b) **WAJIB** Relational model sebagai desain logis
  - (c) **WAJIB** Reduksi E-R ke skema relasional
  - (d) **PAHAM** Perluasan relasional oleh semi-structured dan object-based model

### 6.2 Penjelasan Konsep

#### 6.2.1 Konsep Dasar Model Data

##### Inti Konsep.

Model data adalah sekumpulan alat konseptual untuk mendeskripsikan data, relasi antar data, semantik data, dan batasan konsistensinya. Konsep ini penting karena database harus dirancang berdasarkan makna informasi organisasi, bukan hanya berdasarkan tabel yang kebetulan mudah dibuat.

Motivasi pemodelan data adalah mengoptimalkan cara informasi disimpan dan digunakan. Organisasi memiliki banyak objek dan peristiwa: mahasiswa mengambil mata kuliah, instructor mengajar course, departemen menawarkan program, pelanggan melakukan transaksi. Model data



membantu menentukan objek mana yang perlu direpresentasikan, hubungan apa yang penting, dan aturan apa yang harus dijaga.

Mekanisme dasar pemodelan data:

1. Identifikasi grup data utama yang relevan dengan organisasi.
2. Definisikan konten rinci atau atribut dari tiap grup.
3. Tentukan hubungan antar grup data.
4. Tentukan batasan konsistensi, seperti key, cardinality, atau participation.
5. Terjemahkan model konseptual ke struktur logis yang dapat diimplementasikan.

Dengan prosedur ini, pemodelan data menjadi jembatan antara kebutuhan pengguna dan schema database. Tanpa pemodelan, desain tabel mudah menjadi ad hoc dan rentan redundansi.

### 6.2.2 Level Pemodelan Data

#### Inti Konsep.

Pemodelan data bergerak dari conceptual model yang independen teknologi, ke logical model yang siap diterjemahkan menjadi struktur database, lalu physical model yang memperhatikan penyimpanan dan performa. Pemisahan level ini penting agar kebutuhan bisnis tidak tercampur terlalu awal dengan detail implementasi.

Motivasi level pemodelan adalah separation of concerns. Pada awal desain, pengguna domain perlu mendiskusikan makna data tanpa terganggu detail indeks, blok disk, atau sintaks SQL. Setelah makna data jelas, perancang baru menentukan struktur logis dan fisik.

| Level                        | Definisi dan Peran                                                                                                      |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Conceptual data model</b> | Spesifikasi tingkat tinggi yang independen dari teknologi. Dipakai untuk memahami kebutuhan awal, misalnya E-R diagram. |
| <b>Logical data model</b>    | Struktur logis yang siap diimplementasikan ke database, misalnya skema relasi, tabel, kolom, dan key.                   |
| <b>Physical data model</b>   | Organisasi data pada sistem penyimpanan aktual, termasuk struktur akses, indeks, dan pertimbangan performa.             |

Tabel 10: Level pemodelan data

Contoh alur: kebutuhan “mahasiswa mengambil mata kuliah” pertama dimodelkan sebagai entity **Student**, entity **Course**, dan relationship **Takes**. Setelah itu, logical model menerjemahkannya menjadi tabel **student**, **course**, dan **takes**. Physical model kemudian menentukan indeks pada **student.ID** atau **takes.course\_id**.

Level ini berhubungan dengan data abstraction pada pengantar database: conceptual/logical/physical modeling membantu merancang struktur pada level yang tepat.

### 6.2.3 Entity-Relationship Model

#### Inti Konsep.

Entity-Relationship Model adalah model konseptual tingkat tinggi yang merepresentasikan dunia nyata sebagai entitas, atribut, dan relasi. Model ini penting karena membantu pengguna dan perancang menyepakati makna data sebelum data diterjemahkan menjadi tabel.

Motivasi E-R model adalah kebutuhan menangkap semantik dunia nyata secara visual dan mudah didiskusikan. Pengguna domain sering lebih mudah memahami diagram entitas dan hubungan daripada schema SQL.

| Komponen             | Definisi                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------------|
| <b>Entity</b>        | Objek dunia nyata yang dapat dibedakan dari objek lain, misalnya student, instructor, course. |
| <b>Attribute</b>     | Properti yang mendeskripsikan entity, misalnya ID, name, salary.                              |
| <b>Relationship</b>  | Asosiasi antar entity, misalnya instructor menjadi advisor dari student.                      |
| <b>Key</b>           | Atribut atau himpunan atribut yang mengidentifikasi entity secara unik.                       |
| <b>Cardinality</b>   | Batas jumlah keterhubungan antar entity, seperti one-to-one, one-to-many, atau many-to-many.  |
| <b>Participation</b> | Menyatakan apakah semua entity wajib berpartisipasi dalam relationship atau hanya sebagian.   |

Tabel 11: Komponen dasar E-R model

Contoh dari sumber adalah diagram konseptual universitas dengan entity seperti **instructor**, **student**, dan relationship **advisor**. E-R model ini nantinya direduksi ke skema relasional agar dapat diimplementasikan dalam SQL.

#### 6.2.4 Relational Model

##### Inti Konsep.

Relational model mendeskripsikan data sebagai sekumpulan tabel dengan atribut bernama dan tuple sebagai baris data. Model ini penting karena menjadi model logis utama DBMS relasional dan dasar bagi SQL, aljabar relasional, serta normalisasi.

Motivasi relational model adalah menyediakan struktur logis yang sederhana, matematis, dan dapat dimanipulasi dengan bahasa formal. Setelah E-R model menangkap kebutuhan konseptual, relational model membuatnya siap diimplementasikan sebagai tabel.

Skema relasi ditulis:

$$R(A_1, A_2, \dots, A_n) \quad (2)$$

dengan  $R$  nama relasi dan  $A_i$  atribut. Contoh dari sumber:

$$\text{instructor}(ID, \text{name}, \text{dept\_name}, \text{salary})$$

Satu tuple aktual dapat berisi nilai seperti (22222, *Einstein*, *Physics*, 95000).

| Istilah   | Makna                                  |
|-----------|----------------------------------------|
| Relation  | Tabel secara logis.                    |
| Attribute | Kolom bernama dalam relasi.            |
| Tuple     | Baris data dalam relasi.               |
| Domain    | Himpunan nilai yang sah untuk atribut. |
| Schema    | Struktur relasi.                       |
| Instance  | Isi aktual relasi pada satu waktu.     |

Tabel 12: Istilah dasar relational model

Relational model berhubungan langsung dengan relational database design. Setelah tabel terbentuk, FD dan normalisasi dipakai untuk mengevaluasi apakah struktur tabel tersebut bebas anomali.

### 6.2.5 Semi-Structured dan Object-Based Data Model

#### Inti Konsep.

Semi-structured data model memberi fleksibilitas ketika data sejenis tidak selalu memiliki atribut yang sama, sedangkan object-based data model menggabungkan data dan operasi dalam konsep objek. Keduanya penting sebagai perluasan ketika model relasional yang kaku tidak cukup ekspresif.

Motivasi semi-structured model adalah banyak data modern tidak rapi seperti tabel. Dokumen JSON atau XML dapat memiliki field berbeda antar item. Model relasional mewajibkan tuple dalam satu tabel mengikuti atribut yang sama, sedangkan model semi-structured lebih fleksibel.

Contoh semi-structured data:

Listing 2: Contoh data semi-terstruktur JSON

```
1 {  
2   "student_id": "13524044",  
3   "name": "Narendra",  
4   "contacts": {  
5     "email": "student@example.com"  
6   }  
7 }
```

Object-based data model mengadaptasi paradigma object-oriented. Data dan operasi dibungkus ke dalam objek. Ketika digabung dengan relasional, object-relational model dapat menambahkan inheritance, subtype, dan tipe data kompleks.

Hubungannya dengan relational model adalah sebagai perluasan. Relational model tetap kuat untuk data terstruktur dan query formal, tetapi semi-structured dan object-based model membantu menangani variasi struktur dan objek kompleks.

### 6.2.6 Network dan Hierarchical Model

#### Inti Konsep.

Network model dan hierarchical model adalah model klasik berbasis record dan link. Keduanya penting secara historis karena menunjukkan pendekatan sebelum relational model menjadi dominan.

Network model merepresentasikan data sebagai struktur graf. Record types digambarkan sebagai kotak, sedangkan links menghubungkan record types. Karakteristik pentingnya adalah child dapat memiliki banyak parent, sehingga mendukung relasi many-to-many, many-to-one, dan one-to-one.

Hierarchical model mirip network model, tetapi strukturnya dibatasi menjadi pohon berakar. Setiap child hanya memiliki satu parent. Struktur ini sederhana, tetapi kurang fleksibel untuk relasi many-to-many.

| Aspek         | Network Model                                    | Hierarchical Model                        |
|---------------|--------------------------------------------------|-------------------------------------------|
| Struktur      | Graf dengan record types dan links.              | Pohon berakar.                            |
| Parent child  | Child dapat memiliki banyak parent.              | Child hanya memiliki satu parent.         |
| Kardinalitas  | Mendukung many-to-many, many-to-one, one-to-one. | Lebih cocok untuk one-to-many bertingkat. |
| Fleksibilitas | Lebih fleksibel, tetapi navigasi kompleks.       | Lebih sederhana, tetapi kaku.             |

Tabel 13: Network model dan hierarchical model

Keduanya berhubungan dengan relational model sebagai pembanding. Relational model menghindari navigasi pointer eksplisit dan menyediakan query berbasis nilai serta operasi relasional.

### 6.2.7 Hubungan Antar Model

#### Inti Konsep.

Model data saling berhubungan sebagai tahapan dan alternatif representasi. Dalam desain database relasional, E-R model biasanya dipakai untuk desain konseptual, lalu direduksi menjadi relational model untuk implementasi logis.

Mekanisme transisi umum:

1. Kebutuhan pengguna dianalisis.
2. E-R model menangkap entity, relationship, attribute, key, cardinality, dan participation.
3. E-R diagram direduksi menjadi skema relasional.
4. SQL dipakai untuk mendefinisikan tabel, key, dan constraint.
5. Normalisasi mengevaluasi kualitas skema relasional.

Semi-structured dan object-based model tidak selalu menggantikan relational model, tetapi memperluas pilihan ketika data tidak cocok dengan tabel kaku. Network dan hierarchical model membantu memahami sejarah model data dan keterbatasan struktur navigasional.

## 6.3 Algoritma dan Rumus Penting

### 6.3.1 Prosedur Pemodelan Data

#### Tujuan Algoritma.

Prosedur ini membantu mengubah kebutuhan informasi organisasi menjadi struktur data yang dapat dirancang lebih lanjut.

Listing 3: Pseudocode pemodelan data awal

```
1 Input: organizational information requirements
2 Output: conceptual data model
3
4 identify main data groups
5 for each data group do
6     define detailed attributes and meanings
7 identify relationships among data groups
8 define keys and consistency constraints
9 validate model with stakeholders
10 prepare logical design from conceptual model
```

### 6.3.2 Transisi E-R ke Relasional

#### Makna Prosedur.

Transisi ini menjelaskan posisi E-R dan relational model dalam desain database: E-R menangkap makna, relational model menyiapkan implementasi.

Listing 4: Pseudocode transisi model konseptual ke logis

```
1 Input: E-R diagram
2 Output: relational schemas
3
4 for each entity set do
5     create a relation schema with its attributes
6     choose primary key
7 for each relationship set do
8     map the relationship according to cardinality
9     add foreign keys or create a new relation as needed
10 for each constraint do
11     translate it into keys, foreign keys, or checks when possible
```

## 7 Relational Model

Relational Model adalah model logis utama dalam DBMS relasional. Model ini merepresentasikan data sebagai relasi atau tabel berbasis himpunan, dengan atribut sebagai kolom dan tuple sebagai baris. Kekuatan model relasional terletak pada kesederhanaan struktur tabel dan fondasi matematis yang memungkinkan query formal, constraint, dan normalisasi didefinisikan secara presisi.

Fokus utama untuk ujian adalah memahami anatomi relasi, domain dan nilai atomik, makna NULL, schema-instance, key, foreign key, schema diagram, view, dan hubungan model relasional dengan SQL serta relational database design.

### 7.1 Outline Fundamental Konsep Materi

1. **WAJIB** Konsep dasar relasi
  - (a) **WAJIB** Relation sebagai tabel berbasis set
  - (b) **WAJIB** Attribute/column
  - (c) **WAJIB** Tuple/row
  - (d) **WAJIB** Sifat unordered dan tanpa tuple duplikat
2. **WAJIB** Domain, atomic value, dan NULL
  - (a) **WAJIB** Domain
  - (b) **WAJIB** Atomic value dan 1NF
  - (c) **WAJIB** Tiga makna NULL
  - (d) **WAJIB** Aturan NULL  $\neq$  NULL
3. **WAJIB** Schema dan instance
  - (a) **WAJIB** Relation schema  $R(A_1, \dots, A_n)$
  - (b) **WAJIB** Database schema
  - (c) **WAJIB** Relation/database instance
4. **WAJIB** Keys
  - (a) **WAJIB** Superkey
  - (b) **WAJIB** Candidate key
  - (c) **WAJIB** Primary key
  - (d) **WAJIB** Foreign key
5. **PAHAM** Representasi dan penerapan
  - (a) **PAHAM** Schema diagram
  - (b) **PAHAM** View sebagai tabel virtual
6. **WAJIB** Hubungan dengan materi lain
  - (a) **WAJIB** Relational algebra dan calculus
  - (b) **WAJIB** SQL
  - (c) **WAJIB** Normalisasi dan relational database design

### 7.2 Penjelasan Konsep

#### 7.2.1 Relation, Attribute, dan Tuple

### Inti Konsep.

Relasi adalah tabel logis berbasis teori himpunan; atribut adalah kolom bernama; tuple adalah baris data dalam relasi. Konsep ini penting karena seluruh operasi relasional, SQL, dan normalisasi beroperasi pada struktur dasar ini.

Motivasi relational model adalah menyediakan representasi data yang sederhana tetapi kuat secara matematis. Daripada menyimpan data sebagai struktur pointer yang harus dinavigasi manual, model relasional menyimpan data sebagai tabel dan menyediakan operasi berbasis nilai.

Secara formal, relasi adalah himpunan tuple. Karena relasi dipandang sebagai himpunan, urutan tuple tidak penting dan tuple duplikat tidak diperbolehkan dalam model matematis murni. Dalam implementasi SQL, hasil query dapat mempertahankan duplikasi karena SQL menggunakan semantik multiset, tetapi konsep teoritis relasi tetap berbasis set.

| Istilah          | Definisi dan Contoh                                                                                                      |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Relation</b>  | Tabel logis bernama unik. Contoh: <code>instructor</code> .                                                              |
| <b>Attribute</b> | Kolom bernama dalam relasi. Contoh: <code>ID</code> , <code>name</code> , <code>dept_name</code> , <code>salary</code> . |
| <b>Tuple</b>     | Baris data dalam relasi. Contoh: $(22222, Einstein, Physics, 95000)$ .                                                   |
| <b>Arity</b>     | Jumlah atribut dalam relasi.                                                                                             |

Tabel 14: Anatomi dasar relasi

Relasi menjadi input dan output operator aljabar relasional. Karena setiap operasi menghasilkan relasi baru, operator dapat dikomposisi menjadi ekspresi query yang kompleks.

### 7.2.2 Domain, Atomic Values, dan Null Values

#### Inti Konsep.

Domain adalah himpunan nilai yang diizinkan untuk sebuah atribut, nilai atomik adalah nilai yang tidak perlu dipecah lagi, dan NULL menyatakan nilai yang tidak diketahui, tidak tersedia, atau tidak berlaku. Konsep ini penting karena validitas tuple bergantung pada nilai atribut yang sesuai domain dan dapat dievaluasi secara logis.

Motivasi domain adalah mencegah atribut menerima nilai sembarang. Atribut `salary` seharusnya menerima angka, bukan gambar; atribut `dept_name` seharusnya menerima nama departemen yang sah.

Secara formal, jika atribut  $A_i$  memiliki domain  $D_i$ , maka nilai tuple pada atribut tersebut harus berada dalam  $D_i$ :

$$t[A_i] \in D_i \quad (3)$$

**Atomic value** berarti nilai atribut tidak dapat atau tidak perlu dipecah lagi oleh model relasional. Satu nomor akun dapat dianggap atomik; satu sel berisi daftar banyak nomor akun tidak atomik. Syarat domain atomik ini menjadi dasar First Normal Form (1NF).

**Null value** adalah nilai khusus yang dapat muncul pada domain untuk menyatakan ketiadaan nilai biasa. Sumber menjelaskan tiga makna null:

1. **Value unknown**: nilai ada tetapi belum diketahui.
2. **Value exists but is not available**: nilai ada tetapi belum tersedia.
3. **Attribute does not apply**: atribut tidak berlaku untuk tuple tersebut.

Aturan penting:

$$NULL \neq NULL \quad (4)$$

Maknanya, dua null tidak boleh dianggap mewakili nilai biasa yang sama. Ketidakpastian ini menyebabkan komplikasi pada operasi perbandingan, constraint, agregasi, dan logika SQL.

### 7.2.3 Schema dan Instance

#### Inti Konsep.

Schema adalah struktur logis relasi atau database, sedangkan instance adalah isi aktual pada satu titik waktu. Perbedaan ini penting karena struktur database relatif stabil, tetapi data berubah setiap saat.

Relation schema dinotasikan:

$$R(A_1, A_2, \dots, A_n) \quad (5)$$

dengan  $R$  sebagai nama relasi dan  $A_1, \dots, A_n$  sebagai atribut. Contoh:

*instructor*(*ID*, *name*, *dept\_name*, *salary*)

**Database schema** adalah kumpulan seluruh relation schema dalam database. Misalnya database universitas dapat memiliki schema untuk **student**, **instructor**, **course**, **department**, **takes**, dan **teaches**.

**Instance** adalah isi aktual dari relasi atau database pada satu waktu. Jika schema adalah desain tabel, instance adalah baris-baris aktual yang sedang tersimpan.

| Konsep            | Makna                                        |
|-------------------|----------------------------------------------|
| Relation schema   | Struktur satu relasi.                        |
| Database schema   | Struktur keseluruhan database.               |
| Relation instance | Isi aktual satu relasi pada satu waktu.      |
| Database instance | Isi aktual seluruh database pada satu waktu. |

Tabel 15: Schema dan instance dalam relational model

Konsep schema-instance dipakai terus dalam SQL. Perintah DDL mengubah schema; perintah DML mengubah instance.

### 7.2.4 Keys

#### Inti Konsep.

Key adalah himpunan atribut yang digunakan untuk mengidentifikasi tuple dan menghubungkan relasi. Konsep ini penting karena tanpa key, tuple sulit dibedakan dan relasi antar tabel tidak dapat dijaga secara aman.

Motivasi key adalah identitas dan referensi. Dalam tabel mahasiswa, setiap mahasiswa harus dapat diidentifikasi secara unik. Dalam tabel pengambilan mata kuliah, nilai mahasiswa harus merujuk mahasiswa yang benar-benar ada.

| Jenis Key            | Definisi                                                                                                 |
|----------------------|----------------------------------------------------------------------------------------------------------|
| <b>Superkey</b>      | Himpunan atribut $K$ yang cukup untuk mengidentifikasi tuple secara unik. Secara FD: $K \rightarrow R$ . |
| <b>Candidate key</b> | Superkey minimal; tidak ada subset sejati dari $K$ yang masih superkey.                                  |
| <b>Primary key</b>   | Candidate key yang dipilih sebagai identifikasi utama tuple.                                             |
| <b>Foreign key</b>   | Atribut pada relasi yang merujuk primary key atau candidate key pada relasi lain.                        |

Tabel 16: Jenis key dalam relational model



Kondisi superkey:

$$\forall t_1, t_2 \in r, t_1[K] = t_2[K] \Rightarrow t_1 = t_2 \quad (6)$$

Foreign key menghubungkan relasi. Jika `course.dept_name` adalah foreign key ke `department.dept_name`, maka setiap nilai `dept_name` pada `course` harus ada pada `department`, kecuali null diizinkan.

Key berhubungan langsung dengan integrity constraints: primary key mendukung entity integrity, foreign key mendukung referential integrity, dan candidate key/superkey menjadi dasar functional dependency pada normalisasi.

### 7.2.5 Schema Diagram dan View

#### Inti Konsep.

Schema diagram adalah representasi visual skema relasional, sedangkan view adalah tabel virtual yang didefinisikan dari query. Keduanya penting untuk memahami struktur database dan menyajikan subset data yang relevan tanpa mengubah tabel dasar.

Schema diagram menggambarkan relasi sebagai kotak berisi atribut. Primary key biasanya digarisbawahi, dan foreign key digambarkan dengan anak panah menuju key relasi yang dirujuk. Diagram ini membantu pembaca melihat struktur dan hubungan antar tabel secara cepat.

View adalah relasi virtual. Ia tidak harus menyimpan data fisik seperti tabel dasar, tetapi dapat didefinisikan sebagai hasil query. View berguna untuk:

1. Menyembunyikan atribut yang tidak relevan atau sensitif.
2. Menyederhanakan query kompleks.
3. Menyajikan derived attribute atau data turunan.
4. Memberi tampilan berbeda untuk pengguna berbeda.

View berhubungan dengan level abstraksi view pada pengantar DBMS dan dengan SQL melalui perintah `create view`.

### 7.2.6 Hubungan Relational Model dengan Materi Lain

#### Inti Konsep.

Relational model menjadi dasar formal bagi relational algebra, SQL, integrity constraints, dan normalisasi. Hampir semua materi setelah model data memakai relasi sebagai struktur input dan output.

Hubungan dengan **Relational Algebra**: operator seperti selection, projection, union, set difference, Cartesian product, dan rename menerima relasi sebagai input dan menghasilkan relasi sebagai output.

Hubungan dengan **SQL**: SQL adalah bahasa praktis untuk mendefinisikan schema relasional, memanipulasi instance, mendeklarasikan key, dan menulis query.

Hubungan dengan **Relational Database Design**: normalisasi menganalisis schema relasional menggunakan functional dependency. Tujuannya adalah mengurangi redundansi dan penggunaan null yang tidak perlu.

## 7.3 Algoritma dan Rumus Penting

### 7.3.1 Relation Schema

#### Makna Rumus.

Relation schema menyatakan struktur sebuah relasi sebagai nama relasi dan daftar atributnya. Rumus ini digunakan untuk mendefinisikan tabel secara logis sebelum diisi tuple.

$$R(A_1, A_2, \dots, A_n) \quad (7)$$

dengan:

- $R$ : nama relasi.
- $A_i$ : atribut ke- $i$ .
- $n$ : arity atau jumlah atribut relasi.

### 7.3.2 Kondisi Superkey

#### Makna Rumus.

Kondisi superkey menyatakan bahwa dua tuple berbeda tidak boleh memiliki nilai yang sama pada atribut key. Rumus ini menjadi dasar primary key dan candidate key.

$$K \text{ superkey untuk } R \iff K \rightarrow R \quad (8)$$

dengan:

- $K$ : himpunan atribut key.
- $R$ : seluruh atribut dalam relasi.
- $K \rightarrow R$ :  $K$  menentukan semua atribut  $R$ .

## 8 Relational Algebra & Calculus

Relational Algebra dan Relational Calculus adalah bahasa kueri formal yang menjadi fondasi matematis SQL. Aljabar relasional bersifat prosedural: ia menjelaskan langkah operasi untuk memperoleh hasil. Kalkulus relasional bersifat deklaratif: ia menjelaskan kondisi data yang diinginkan tanpa menentukan langkah eksekusi.

Fokus utama untuk ujian adalah menguasai operator dasar aljabar relasional, memahami komposisi ekspresi, menerjemahkan kebutuhan query ke notasi formal, memahami division untuk query “untuk semua”, serta memahami hubungan kalkulus dengan SQL subquery seperti `some`, `all`, dan `exists`.

### 8.1 Outline Fundamental Konsep Materi

1. **WAJIB** Bahasa kueri relasional formal
  - (a) **WAJIB** Procedural/imperative
  - (b) **WAJIB** Declarative/non-procedural
  - (c) **PAHAM** Functional
2. **WAJIB** Relational algebra dasar
  - (a) **WAJIB** Closure property
  - (b) **WAJIB** Basic expression
  - (c) **WAJIB** Selection, projection, union, set difference, Cartesian product, rename
  - (d) **WAJIB** Union compatibility
3. **WAJIB** Operator tambahan
  - (a) **WAJIB** Intersection
  - (b) **WAJIB** Theta join, equijoin, natural join
  - (c) **WAJIB** Outer join
  - (d) **PAHAM** Assignment
  - (e) **WAJIB** Division
4. **WAJIB** Extended relational algebra
  - (a) **WAJIB** Generalized projection
  - (b) **WAJIB** Aggregation dan grouping
  - (c) **WAJIB** Null values
  - (d) **PAHAM** Multiset relational algebra
5. **WAJIB** Relational calculus
  - (a) **WAJIB** Tuple Relational Calculus (TRC)
  - (b) **WAJIB** Domain Relational Calculus (DRC)
  - (c) **WAJIB** Quantifier  $\exists$  dan  $\forall$
  - (d) **WAJIB** Safety/domain independence
6. **WAJIB** Hubungan dengan SQL
  - (a) **WAJIB** `some`, `all`, dan `exists`
  - (b) **WAJIB** Relational completeness
  - (c) **WAJIB** Algebra sebagai model evaluasi dan calculus sebagai model deklaratif

## 8.2 Penjelasan Konsep

### 8.2.1 Bahasa Kueri Relasional Formal

#### Inti Konsep.

Bahasa kueri relasional formal adalah bahasa matematis untuk menyatakan query pada relasi. Aljabar relasional menjelaskan bagaimana data dihitung, sedangkan kalkulus relasional menjelaskan data apa yang memenuhi kondisi tertentu.

Motivasi bahasa formal adalah memberi dasar teoretis bagi bahasa praktis seperti SQL. Dengan bahasa formal, kita dapat mendefinisikan arti query secara presisi, membuktikan ekuivalensi query, dan memahami optimasi query.

| Kategori                     | Makna                                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------------|
| Procedural / imperative      | Pengguna menyebut langkah untuk memperoleh data. Contoh: relational algebra.                  |
| Declarative / non-procedural | Pengguna menyebut kondisi data yang diinginkan. Contoh: tuple dan domain relational calculus. |
| Functional                   | Query dipandang sebagai fungsi matematis dari input relasi ke output relasi.                  |

Tabel 17: Kategori bahasa kueri formal

SQL mengambil gaya deklaratif dari kalkulus, tetapi DBMS mengeksekusinya dengan rencana operasi yang mirip aljabar relasional.

### 8.2.2 Relational Algebra Dasar

#### Inti Konsep.

Relational algebra adalah bahasa prosedural yang memakai operator pada relasi. Setiap operator menerima satu atau dua relasi dan menghasilkan relasi baru, sehingga ekspresi dapat dikomposisi.

**Closure property** berarti hasil setiap operasi aljabar relasional selalu berupa relasi. Karena output adalah relasi, hasil operasi dapat menjadi input operasi berikutnya.

**Basic expression** dapat berupa relasi aktual dalam database atau relasi konstan. Dari ekspresi dasar ini, operator membentuk ekspresi yang lebih kompleks.

| Operator          | Notasi                     | Makna                                                                  |
|-------------------|----------------------------|------------------------------------------------------------------------|
| Selection         | $\sigma_p(r)$              | Memilih tuple dalam $r$ yang memenuhi predikat $p$ .                   |
| Projection        | $\Pi_{A_1, \dots, A_k}(r)$ | Memilih atribut tertentu dan menghapus duplikasi dalam model set.      |
| Union             | $r \cup s$                 | Menggabungkan tuple dari $r$ atau $s$ . Relasi harus union-compatible. |
| Set difference    | $r - s$                    | Mengambil tuple yang ada di $r$ tetapi tidak ada di $s$ .              |
| Cartesian product | $r \times s$               | Menggabungkan setiap tuple $r$ dengan setiap tuple $s$ .               |
| Rename            | $\rho_x(E)$                | Memberi nama baru pada hasil ekspresi $E$ .                            |

Tabel 18: Enam operator dasar relational algebra

Union compatibility mensyaratkan dua relasi memiliki arity sama dan domain atribut yang bersesuaian kompatibel. Tanpa syarat ini, operasi himpunan seperti union, intersection, dan difference tidak bermakna.

Contoh komposisi:

$$\Pi_{name}(\sigma_{dept\_name="Physics"}(instructor))$$

Ekspresi ini memilih instructor dari departemen Physics, lalu memproyeksikan atribut **name**.

### 8.2.3 Operator Tambahan

#### Inti Konsep.

Operator tambahan seperti intersection, join, outer join, assignment, dan division tidak selalu menambah kekuatan ekspresif, tetapi membuat query umum jauh lebih ringkas. Operator ini penting karena banyak query nyata lebih mudah ditulis sebagai join atau division daripada kombinasi operator dasar mentah.

| Operator     | Notasi                              | Makna                                                                                 |
|--------------|-------------------------------------|---------------------------------------------------------------------------------------|
| Intersection | $r \cap s$                          | Tuple yang muncul di $r$ dan $s$ ; dapat ditulis $r - (r - s)$ .                      |
| Theta join   | $r \bowtie_{\theta} s$              | Join dengan predikat $\theta$ ; setara $\sigma_{\theta}(r \times s)$ .                |
| Equijoin     | $r \bowtie_{A=B} s$                 | Theta join dengan predikat kesamaan.                                                  |
| Natural join | $r \bowtie s$                       | Join otomatis pada atribut bernama sama dan menghapus kolom ganda.                    |
| Outer join   | <i>left, right, full outer join</i> | Join yang mempertahankan tuple tidak cocok dengan mengisi nilai null.                 |
| Assignment   | $\leftarrow$                        | Menyimpan hasil ekspresi ke variabel sementara.                                       |
| Division     | $r \nabla \cdot s$                  | Menjawab query “untuk semua”, misalnya mahasiswa yang mengambil semua course Biology. |

Tabel 19: Operator tambahan relational algebra

Natural join bersifat komutatif dan asosiatif dalam kondisi atribut yang sesuai:

$$r \bowtie s = s \bowtie r$$

$$(r \bowtie s) \bowtie t = r \bowtie (s \bowtie t)$$

Outer join penting ketika tuple tanpa pasangan tidak boleh hilang. Misalnya left outer join mempertahankan semua tuple relasi kiri, lalu mengisi atribut relasi kanan dengan null jika tidak ada kecocokan.

### 8.2.4 Division

#### Inti Konsep.

Division adalah operator untuk query yang membutuhkan pemenuhan terhadap semua nilai dalam relasi pembagi. Operator ini penting karena banyak pertanyaan database berbentuk “temukan X yang berhubungan dengan semua Y”.

Definisi formal: diberikan  $r(R)$  dan  $s(S)$  dengan  $S \subseteq R$ . Hasil  $r \nabla \cdot s$  adalah relasi terbesar  $t$  dengan skema  $R - S$  sedemikian sehingga:

$$t \times s \subseteq r \quad (9)$$

Contoh sumber: cari mahasiswa yang mengambil semua course yang ditawarkan departemen Biology.

$$r(ID, course\_id) = \Pi_{ID, course\_id}(takes)$$

$$s(course\_id) = \Pi_{course\_id}(\sigma_{dept\_name="Biology"}(course))$$

Maka  $r \nabla \cdot s$  menghasilkan ID mahasiswa yang mengambil seluruh course Biology.

Dekomposisi division ke operator dasar:

$$T_1 = \Pi_{R-S}(r) \quad (10)$$

$$T_2 = \Pi_{R-S}((T_1 \times s) - r) \quad (11)$$

$$r \nabla \cdot s = T_1 - T_2 \quad (12)$$

Maknanya:  $T_1$  adalah semua kandidat.  $T_1 \times s$  adalah semua pasangan yang seharusnya ada jika kandidat memenuhi semua syarat. Setelah dikurangi  $r$ , terlihat pasangan yang gagal. Kandidat gagal dibuang dari  $T_1$ .

### 8.2.5 Extended Relational Algebra

#### Inti Konsep.

Extended relational algebra menambahkan operasi praktis seperti generalized projection, aggregation, grouping, null handling, dan multiset semantics. Konsep ini penting karena SQL dunia nyata membutuhkan perhitungan aritmatika, agregasi, dan duplikasi yang tidak seluruhnya nyaman dalam aljabar set murni.

**Generalized projection** mengizinkan ekspresi aritmatika pada daftar proyeksi:

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

dengan  $F_i$  adalah ekspresi aritmatika. Contoh:  $\Pi_{ID, name, salary/12}(instructor)$  untuk menampilkan gaji bulanan.

**Aggregation dan grouping** menghitung fungsi statistik seperti avg, min, max, sum, dan count. Contoh rata-rata gaji per departemen:

$$dept\_name \mathcal{G}_{avg(salary)}(instructor)$$

**Null values** menyatakan unknown atau nilai tidak eksis. Operasi aritmatika dengan null menghasilkan null. Fungsi agregasi umumnya mengabaikan null, tetapi untuk grouping dan eliminasi duplikasi, dua null dapat diperlakukan sebagai nilai yang sama.

**Multiset relational algebra** memodelkan SQL yang mempertahankan duplikasi. Aturan umum:

- Selection mempertahankan jumlah salinan tuple yang lolos.
- Projection menghasilkan tuple untuk setiap tuple input, termasuk duplikasi.
- Cartesian product menghasilkan  $m \times n$  salinan jika tuple kiri memiliki  $m$  salinan dan tuple kanan memiliki  $n$  salinan.
- Union menghasilkan  $m + n$ , intersection menghasilkan  $\min(m, n)$ , difference menghasilkan  $\max(0, m - n)$ .

### 8.2.6 Relational Calculus

#### Inti Konsep.

Relational calculus adalah bahasa kueri deklaratif berbasis logika predikat. TRC memakai variabel tuple, sedangkan DRC memakai variabel domain; keduanya menyatakan kondisi hasil, bukan langkah eksekusi.

Tuple Relational Calculus (TRC) memiliki bentuk:

$$\{t \mid P(t)\} \quad (13)$$

yang berarti himpunan tuple  $t$  sehingga predikat  $P(t)$  benar.

Contoh:

$$\{t \mid t \in \text{instructor} \wedge t[\text{salary}] > 80000\}$$

Domain Relational Calculus (DRC) memiliki bentuk:

$$\{\langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n)\} \quad (14)$$

yang berarti hasil dibentuk dari variabel domain individual, bukan tuple utuh.

Contoh:

$$\{\langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in \text{instructor} \wedge s > 80000\}$$

Kalkulus memakai quantifier  $\exists$  dan  $\forall$ .  $\exists x$  berarti ada setidaknya satu nilai  $x$  yang memenuhi predikat.  $\forall x$  berarti semua nilai  $x$  dalam domain relevan memenuhi predikat.

## 8.2.7 Safety, SQL Quantifier, dan Relational Completeness

### Inti Konsep.

Ekspresi kalkulus harus aman agar hasilnya terbatas dan dapat dihitung. SQL mengadopsi gagasan quantifier melalui **some**, **all**, dan **exists**; bahasa dikatakan relationally complete jika mampu mengekspresikan semua query kalkulus relasional yang aman.

Safety atau domain independence diperlukan karena kalkulus dapat menulis ekspresi yang menghasilkan relasi tak hingga. Contoh tidak aman:

$$\{t \mid \neg(t \in r)\}$$

Ekspresi ini berarti semua tuple di semesta yang bukan anggota  $r$ , sehingga hasilnya tidak terbatas.

Operator SQL terkait quantifier:

| Operator      | Makna Logis                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------|
| <b>some</b>   | $F < \text{comp} > \text{some } r \Leftrightarrow \exists t \in r \text{ sehingga } F < \text{comp} > t.$ |
| <b>all</b>    | $F < \text{comp} > \text{all } r \Leftrightarrow \forall t \in r, F < \text{comp} > t.$                   |
| <b>exists</b> | $\text{exists } r \Leftrightarrow r \neq \emptyset.$                                                      |

Tabel 20: Quantifier kalkulus dalam SQL

Catatan penting dari sumber: **= some** ekuivalen dengan **in**, tetapi **<> some** tidak ekuivalen dengan **not in**. Sebaliknya, **<> all** ekuivalen dengan **not in**.

Relational completeness menyatakan kemampuan bahasa mengekspresikan semua query yang dapat ditulis dalam kalkulus relasional aman. Aljabar relasional, TRC, dan DRC secara teori memiliki kekuatan ekspresif yang setara untuk query relasional aman.

## 8.3 Algoritma dan Rumus Penting

### 8.3.1 Division

#### Tujuan Algoritma.

Algoritma division menemukan kandidat yang berhubungan dengan semua nilai dalam relasi pembagi. Ini adalah pola query “untuk semua”.

Listing 5: Pseudocode division relational algebra

```
1 Input: r(R), s(S) with S subset of R
2 Output: r_div_s on schema R - S
3
4 T1 := project R-S from r
5 T2 := project R-S from ((T1 cross s) - r)
6 result := T1 - T2
7 return result
```

### 8.3.2 Selection dan Projection

#### Makna Rumus.

Selection memilih baris berdasarkan predikat, sedangkan projection memilih kolom tertentu. Keduanya adalah operator paling dasar untuk membentuk query.

$$\sigma_p(r) = \{t \mid t \in r \wedge p(t)\} \quad (15)$$

$$\Pi_{A_1, \dots, A_k}(r) \quad (16)$$

dengan:

- $p$ : predikat seleksi.
- $r$ : relasi input.
- $A_1, \dots, A_k$ : atribut yang dipertahankan.



## 9 SQL (Structured Query Language)

SQL adalah bahasa standar untuk mendefinisikan, mengambil, memanipulasi, mengamankan, dan mengontrol transaksi pada database relasional. SQL menghubungkan teori model relasional dengan praktik DBMS: struktur tabel dibuat dengan DDL, data diambil dan dimodifikasi dengan DML, constraint ditegakkan dengan deklarasi skema, dan query deklaratif diterjemahkan DBMS menjadi rencana eksekusi.

Fokus utama untuk ujian adalah menguasai struktur query `select-from-where`, DDL dan constraint, NULL dan three-valued logic, agregasi, grouping, subquery, joins, views, modifikasi data, authorization, transactions, dan hubungan SQL dengan relational algebra/calculus.

### Komentar 9.1: Keterbatasan Sumber

Query mendalam NotebookLM untuk SQL beberapa kali terpotong. Bagian ini disusun dari hasil query SQL bagian 1 yang lengkap, outline SQL NotebookLM yang sudah disetujui, dan pengetahuan umum untuk melengkapi bagian yang terpotong. Pelengkap umum yang tidak eksplisit di hasil query dibungkus dalam environment `cmt`.

### 9.1 Outline Fundamental Konsep Materi

1. **WAJIB** Latar belakang dan bagian SQL
  - (a) **PAHAM** SEQUEL, System R, ANSI/ISO
  - (b) **WAJIB** DDL, DML, view definition, transaction control, embedded/dynamic SQL, authorization
2. **WAJIB** DDL dan constraints
  - (a) **WAJIB** `create table`, `drop table`, `alter table`
  - (b) **WAJIB** Tipe data `char`, `varchar`, `numeric`
  - (c) **WAJIB** `primary key`, `foreign key`, `not null`, `unique`, `check`
3. **WAJIB** Query dasar DML
  - (a) **WAJIB** `select-from-where`
  - (b) **WAJIB** `distinct/all`, `alias as`, `predicates`, `between`, `tuple comparison`
  - (c) **WAJIB** `order by`
  - (d) **WAJIB** String matching dengan `like`
4. **WAJIB** NULL dan logika SQL
  - (a) **WAJIB** Three-valued logic
  - (b) **WAJIB** `case` dan `coalesce`
5. **WAJIB** Query menengah
  - (a) **WAJIB** Set operations
  - (b) **WAJIB** Aggregate functions, `group by`, `having`
  - (c) **WAJIB** Nested subqueries
  - (d) **WAJIB** Join expressions
  - (e) **WAJIB** Views
  - (f) **WAJIB** `insert`, `delete`, `update`
6. **PAHAM** SQL lanjutan

- (a) **PAHAM** Authorization dan roles
- (b) **PAHAM** Transactions
- (c) **PAHAM** JDBC/ODBC, stored procedure, trigger, prepared statement
- (d) **PAHAM** Recursive query, window/ranking, cube, rollup

## 9.2 Penjelasan Konsep

### 9.2.1 Latar Belakang dan Bagian SQL

#### Inti Konsep.

SQL adalah bahasa komprehensif untuk database relasional, bukan hanya bahasa query. Ia mencakup definisi skema, manipulasi data, view, transaksi, embedding ke bahasa pemrograman, dan authorization.

SQL berawal dari bahasa SEQUEL yang dikembangkan IBM pada proyek System R di San Jose Research Laboratory. Namanya kemudian menjadi SQL. Standarisasi ANSI/ISO menghasilkan versi seperti SQL-86, SQL-89, SQL-92, SQL:1999, dan SQL:2003.

| Bagian SQL           | Peran                                                                     |
|----------------------|---------------------------------------------------------------------------|
| DDL                  | Mendefinisikan dan memodifikasi schema, tabel, tipe data, dan constraint. |
| DML                  | Mengambil, menyisipkan, menghapus, dan memperbarui tuple.                 |
| View definition      | Mendefinisikan tabel virtual berbasis query.                              |
| Transaction control  | Mengatur commit dan rollback transaksi.                                   |
| Embedded/dynamic SQL | Menyisipkan SQL ke bahasa pemrograman umum.                               |
| Authorization        | Mengatur hak akses pengguna terhadap relasi dan view.                     |

Tabel 21: Bagian utama SQL

SQL berhubungan dengan relational algebra karena **select**, **from**, dan **where** berkorelasi dengan projection, Cartesian product/join, dan selection. SQL juga berhubungan dengan relational calculus karena pengguna mendeklarasikan hasil yang diinginkan, bukan algoritma fisiknya.

### 9.2.2 DDL dan Constraints

#### Inti Konsep.

DDL digunakan untuk mendefinisikan struktur database: tabel, atribut, tipe data, dan constraint. Konsep ini penting karena kualitas data tidak hanya ditentukan oleh query, tetapi oleh aturan yang tertanam pada schema.

Perintah DDL utama:

Listing 6: Contoh DDL SQL

```

1 create table instructor (
2     ID varchar(5),
3     name varchar(20) not null,
4     dept_name varchar(20),
5     salary numeric(8,2),
6     primary key (ID),
7     foreign key (dept_name) references department,
8     check (salary > 29000)
9 );

```

```

10
11 alter table instructor add office varchar(10);
12 drop table instructor;

```

`create table` membuat schema relasi. `drop table` menghapus data sekaligus definisi schema. `alter table` memodifikasi schema; ketika atribut baru ditambahkan, nilai awalnya dapat menjadi null.

| Tipe/Constraint | Makna                                                                 |
|-----------------|-----------------------------------------------------------------------|
| char(n)         | String panjang tetap.                                                 |
| varchar(n)      | String panjang bervariasi sampai $n$ .                                |
| numeric(p,d)    | Angka dengan total $p$ digit dan $d$ digit di belakang desimal.       |
| primary key     | Unik dan non-null.                                                    |
| foreign key     | Merujuk key pada relasi lain.                                         |
| not null        | Melarang nilai null pada atribut.                                     |
| unique          | Memaksa nilai unik, tetapi dapat memperbolehkan null tergantung DBMS. |
| check(P)        | Memaksa tuple memenuhi predikat $P$ .                                 |

Tabel 22: Tipe data dan constraint SQL

DDL berkaitan langsung dengan integrity constraints. Primary key menerapkan entity integrity; foreign key menerapkan referential integrity; check dan not null menerapkan domain atau attribute constraints.

### 9.2.3 Query Dasar Select-From-Where

#### Inti Konsep.

Struktur dasar query SQL adalah `select-from-where`: `select` memilih atribut, `from` memilih relasi sumber, dan `where` memfilter tuple. Pola ini penting karena menjadi bentuk dasar hampir semua query SQL.

Bentuk umum:

Listing 7: Struktur dasar query SQL

```

1 select A1, A2, ..., An
2 from r1, r2, ..., rm
3 where P;

```

Hubungan dengan aljabar relasional:

$$\Pi_{A_1, \dots, A_n}(\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$

Fitur penting:

- `distinct`: menghapus duplikasi hasil.
- `all`: mempertahankan duplikasi secara eksplisit.
- `as`: memberi alias atribut atau tabel.
- `between`: predikat rentang.
- Tuple comparison: membandingkan beberapa atribut sekaligus.
- `order by`: mengurutkan hasil, default `asc`, dapat memakai `desc`.

Contoh:

Listing 8: Contoh query dasar

```
1 select distinct name
2 from instructor
3 where salary between 90000 and 100000
4 order by name asc;
```

String matching memakai `like`. Karakter `%` mencocokkan substring bebas, sedangkan `_` mencocokkan tepat satu karakter.

#### 9.2.4 NULL dan Three-Valued Logic

##### Inti Konsep.

NULL menyatakan nilai yang tidak diketahui, tidak tersedia, atau tidak berlaku. SQL memakai three-valued logic karena perbandingan dengan null tidak menghasilkan true atau false, melainkan unknown.

Motivasi three-valued logic adalah nilai tidak diketahui tidak boleh diperlakukan sebagai nilai biasa. Jika `salary` bernilai null, predikat `salary > 50000` tidak dapat dipastikan benar atau salah.

| Ekspresi          | Hasil   |
|-------------------|---------|
| true and unknown  | unknown |
| false and unknown | false   |
| unknown or true   | true    |
| unknown or false  | unknown |

Tabel 23: Contoh three-valued logic

Klausula `where` hanya mempertahankan tuple yang predikatnya true. Jika hasilnya unknown, tuple tidak dipilih.

`case` dan `coalesce` membantu menangani null:

Listing 9: Case dan coalesce

```
1 select case
2     when sum(credits) is not null then sum(credits)
3     else 0
4     end as total_credits
5 from takes;
6
7 select coalesce(sum(credits), 0) as total_credits
8 from takes;
```

#### 9.2.5 Set Operations, Aggregation, dan Grouping

##### Inti Konsep.

Set operations menggabungkan hasil beberapa query, sedangkan aggregation dan grouping merangkum banyak tuple menjadi nilai statistik. Konsep ini penting karena SQL sering dipakai untuk laporan, rekap, dan analisis data.

Set operations mencakup `union`, `intersect`, dan `except`. Operasi ini berhubungan langsung dengan  $\cup$ ,  $\cap$ , dan  $-$  pada aljabar relasional.

Aggregate functions:

- `avg`: rata-rata.

- **min**: nilai minimum.
- **max**: nilai maksimum.
- **sum**: jumlah.
- **count**: jumlah tuple atau nilai.

Contoh:

Listing 10: Aggregation dan grouping

```
1 select dept_name, avg(salary) as avg_salary
2 from instructor
3 group by dept_name
4 having avg(salary) > 70000;
```

**where** memfilter tuple sebelum grouping, sedangkan **having** memfilter grup setelah agregasi.

### 9.2.6 Nested Subqueries

#### Inti Konsep.

Nested subquery adalah query yang berada di dalam query lain. Konsep ini penting karena SQL dapat menyatakan kondisi berbasis himpunan seperti membership, comparison, existence, dan query “untuk semua”.

| Bentuk                   | Makna                                                            |
|--------------------------|------------------------------------------------------------------|
| <b>in/not in</b>         | Mengecek apakah nilai ada atau tidak ada dalam hasil subquery.   |
| <b>some/all</b>          | Membandingkan nilai dengan sebagian atau seluruh hasil subquery. |
| <b>exists/not exists</b> | Mengecek apakah hasil subquery kosong atau tidak.                |
| <b>unique</b>            | Mengecek apakah hasil subquery tidak memiliki duplikasi.         |
| Correlated subquery      | Subquery yang merujuk variabel dari query luar.                  |
| Scalar subquery          | Subquery yang harus mengembalikan tepat satu nilai.              |

Tabel 24: Jenis nested subquery

Contoh **exists**:

Listing 11: Contoh correlated subquery

```
1 select S.ID, S.name
2 from student as S
3 where exists (
4     select *
5     from takes as T
6     where T.ID = S.ID
7 );
```

Pola division dalam SQL sering memakai double **not exists**. Gagasannya: pilih kandidat yang tidak memiliki item wajib yang belum dipenuhi.

#### Komentar 9.2: Catatan: Sumber Pengetahuan Umum

Pola umum query “X yang memenuhi semua Y” dalam SQL adalah: tidak ada Y yang tidak dipenuhi oleh X. Ini sesuai logika  $\forall y P(x, y)$  yang dapat ditulis sebagai  $\neg \exists y \neg P(x, y)$ . Dalam SQL, bentuk ini sering memakai nested **not exists**.

### 9.2.7 Join Expressions dan Views

#### Inti Konsep.

Join expressions menggabungkan tabel berdasarkan kondisi keterhubungan, sedangkan view menyimpan definisi query sebagai tabel virtual. Keduanya penting untuk menyederhanakan query kompleks dan mengelola abstraksi skema.

Jenis join:

- **inner join**: hanya tuple yang cocok.
- **natural join**: otomatis mencocokkan atribut bernama sama.
- **left outer join**: mempertahankan semua tuple kiri.
- **right outer join**: mempertahankan semua tuple kanan.
- **full outer join**: mempertahankan tuple dari kedua sisi.

Contoh:

Listing 12: Join expression

```
1 select I.name, D.building
2 from instructor as I
3 join department as D
4 on I.dept_name = D.dept_name;
```

View:

Listing 13: Create view

```
1 create view physics_instructors as
2 select ID, name
3 from instructor
4 where dept_name = 'Physics';
```

View berguna untuk keamanan dan abstraksi, tetapi tidak semua view dapat diperbarui. View sederhana dari satu tabel tanpa agregasi, **distinct**, atau **group by** lebih mungkin updatable; view kompleks biasanya tidak.

### 9.2.8 Modifikasi Data, Authorization, dan Transaction

#### Inti Konsep.

SQL tidak hanya mengambil data, tetapi juga memodifikasi data, mengatur hak akses, dan mengontrol transaksi. Konsep ini penting karena database harus mendukung operasi nyata dengan keamanan dan atomicity.

Modifikasi data:

Listing 14: Insert delete update

```
1 insert into instructor (ID, name, dept_name, salary)
2 values ('99999', 'Ada', 'Comp. Sci.', 90000);
3
4 delete from instructor
5 where ID = '99999';
6
7 update instructor
8 set salary = salary * 1.05
9 where dept_name = 'Comp. Sci.';
```

Authorization:

#### Listing 15: Grant dan revoke

```
1 grant select on instructor to analyst;
2 revoke select on instructor from analyst;
```

Transaction control:

#### Listing 16: Commit dan rollback

```
1 begin transaction;
2 update account set balance = balance - 100 where account_id = 'A';
3 update account set balance = balance + 100 where account_id = 'B';
4 commit;
5 -- atau rollback jika gagal
```

Transaction berhubungan dengan atomicity: dua update transfer harus dianggap satu unit kerja. Jika salah satu gagal, seluruh transaksi harus dibatalkan.

### 9.2.9 SQL Lanjutan

#### Inti Konsep.

SQL lanjutan memperluas SQL untuk integrasi aplikasi, keamanan, logika prosedural, dan analitik. Konsep ini penting karena database modern tidak hanya menyimpan data, tetapi juga menjadi bagian dari aplikasi dan sistem analitik.

Topik lanjutan mencakup embedded/dynamic SQL, JDBC, ODBC, stored procedures, functions, triggers, prepared statements, recursive query, window functions, ranking, `cube`, dan `rollup`.

#### Komentar 9.3: Catatan: Sumber Pengetahuan Umum

**Prepared statement** membantu mencegah SQL injection karena nilai parameter dikirim terpisah dari teks SQL, bukan digabung sebagai string mentah. SQL injection terjadi ketika input pengguna disisipkan langsung ke query sehingga dapat mengubah struktur query.

Analitik SQL seperti window function, ranking, `cube`, dan `rollup` berhubungan dengan data warehouse dan OLAP. Stored procedures dan triggers berhubungan dengan integrity constraints dan aturan bisnis.

## 9.3 Algoritma dan Rumus Penting

### 9.3.1 Urutan Logis Query SQL

#### Tujuan Algoritma.

Urutan logis ini membantu memahami bahwa SQL tidak dievaluasi dari atas ke bawah seperti teksnya. `from` dan `where` secara konseptual diproses sebelum `select`.

#### Listing 17: Urutan logis query SQL

```
1 Input: SQL query
2 Output: result relation
3
4 1. FROM: determine source relations and joins
5 2. WHERE: filter tuples
6 3. GROUP BY: form groups
7 4. HAVING: filter groups
8 5. SELECT: compute output attributes
9 6. DISTINCT: remove duplicates if requested
10 7. ORDER BY: sort final output
```

### 9.3.2 SQL dan Aljabar Relasional

#### Makna Rumus.

Rumus ini menunjukkan padanan query dasar SQL dengan aljabar relasional: `where` menjadi selection, `select` menjadi projection, dan `from` menjadi product atau join.

$$\text{select } A \text{ from } R \text{ where } P \equiv \Pi_A(\sigma_P(R)) \quad (17)$$

dengan:

- $A$ : daftar atribut hasil.
- $R$ : relasi atau hasil join sumber.
- $P$ : predikat filter.



## 10 Database Design using E-R Model

Database Design using E-R Model membahas cara menerjemahkan kebutuhan dunia nyata ke dalam rancangan konseptual database. E-R model memungkinkan perancang mendefinisikan entitas, atribut, relasi, constraint, dan fitur lanjutan seperti specialization dan aggregation sebelum rancangan direduksi menjadi skema relasional.

Fokus utama untuk ujian adalah memahami fase desain database, komponen dasar E-R, cardinality dan participation, weak entity, notasi E-R diagram, fitur extended E-R, isu desain, serta reduksi E-R ke skema relasional.

### 10.1 Outline Fundamental Konsep Materi

1. **WAJIB** Fase desain basis data
  - (a) **WAJIB** Requirement/initial phase
  - (b) **WAJIB** Conceptual design
  - (c) **WAJIB** Logical design
  - (d) **PAHAM** Physical design
2. **WAJIB** Komponen dasar E-R model
  - (a) **WAJIB** Entity dan entity set
  - (b) **WAJIB** Strong entity dan weak entity
  - (c) **WAJIB** Attributes
  - (d) **WAJIB** Relationship dan relationship set
3. **WAJIB** Constraints dalam E-R
  - (a) **WAJIB** Mapping cardinality
  - (b) **WAJIB** Total dan partial participation
  - (c) **WAJIB** Keys
  - (d) **WAJIB** Identifying relationship dan partial key
4. **WAJIB** Representasi visual
  - (a) **WAJIB** E-R diagram notation
  - (b) **PAHAM** UML class diagram
5. **WAJIB** Extended E-R
  - (a) **WAJIB** Specialization dan generalization
  - (b) **WAJIB** Inheritance
  - (c) **WAJIB** Disjoint/overlapping dan total/partial specialization
  - (d) **WAJIB** Aggregation
6. **WAJIB** Reduksi E-R ke skema relasional
  - (a) **WAJIB** Strong entity
  - (b) **WAJIB** Weak entity
  - (c) **WAJIB** Relationship set
  - (d) **WAJIB** Multivalued attribute
  - (e) **PAHAM** Specialization dan aggregation
7. **PAHAM** Isu desain dan hubungan normalisasi

## 10.2 Penjelasan Konsep

### 10.2.1 Fase Desain Basis Data

#### Inti Konsep.

Desain basis data mengubah kebutuhan pengguna menjadi struktur database operasional melalui fase kebutuhan, konseptual, logis, dan fisik. Konsep ini penting karena kesalahan pada fase awal dapat menghasilkan skema relasional yang redundan atau sulit diimplementasikan.

Motivasi fase desain adalah memisahkan pemahaman kebutuhan dari detail implementasi. Pengguna domain memahami aturan dunia nyata, tetapi belum tentu memahami tabel, key, indeks, atau SQL. Perancang perlu menerjemahkan kebutuhan tersebut secara bertahap.

| Fase                        | Tujuan                                                                                              |
|-----------------------------|-----------------------------------------------------------------------------------------------------|
| Initial / requirement phase | Mengkarakterisasi kebutuhan data calon pengguna melalui interaksi dengan pakar domain dan pengguna. |
| Conceptual design           | Menerjemahkan kebutuhan ke skema konseptual, sering menggunakan E-R model.                          |
| Logical design              | Memetakan skema konseptual ke model implementasi, biasanya skema relasional.                        |
| Physical design             | Menentukan struktur penyimpanan fisik, indeks, dan optimasi performa.                               |

Tabel 25: Fase desain basis data

Fase ini berhubungan dengan semua materi desain. E-R model bekerja pada conceptual design; reduksi E-R bekerja pada logical design; indexing dan tuning berada pada physical design.

### 10.2.2 Entity, Attribute, dan Relationship

#### Inti Konsep.

E-R model merepresentasikan dunia nyata sebagai entity, attribute, dan relationship. Konsep ini penting karena perancang perlu menangkap objek, properti, dan asosiasi dunia nyata sebelum membuat tabel.

**Entity** adalah objek dunia nyata yang dapat dibedakan dari objek lain. **Entity set** adalah kumpulan entity bertipe sama. Contoh: **student**, **instructor**, dan **course**.

**Attribute** mendeskripsikan entity. Klasifikasinya:

| Jenis Atribut | Definisi dan Contoh                                                                            |
|---------------|------------------------------------------------------------------------------------------------|
| Simple        | Tidak dapat dipecah lagi, misalnya ID.                                                         |
| Composite     | Dapat dipecah menjadi sub-atribut, misalnya nama lengkap menjadi nama depan dan nama belakang. |
| Single-valued | Memiliki satu nilai untuk satu entity, misalnya nomor KTP.                                     |
| Multivalued   | Dapat memiliki banyak nilai, misalnya nomor telepon.                                           |
| Derived       | Dihitung dari atribut lain, misalnya umur dari tanggal lahir.                                  |

Tabel 26: Klasifikasi atribut E-R

**Relationship** adalah asosiasi antar entity. **Relationship set** adalah kumpulan relationship bertipe sama. Relationship dapat memiliki **descriptive attributes**; contoh sumber menyebut relasi **takes** antara **student** dan **section** dapat memiliki atribut **grade**.

Relationship memiliki **degree**, yaitu jumlah entity set yang berpartisipasi. Relationship biner melibatkan dua entity set. Relationship n-ary melibatkan lebih dari dua. Recursive relationship terjadi ketika entity set berelasi dengan dirinya sendiri; role diperlukan untuk membedakan peran masing-masing partisipan.

### 10.2.3 Cardinality, Participation, dan Keys

#### Inti Konsep.

Cardinality, participation, dan keys adalah constraint pada E-R model. Mereka penting karena menentukan berapa banyak entity dapat berhubungan, apakah partisipasi wajib, dan bagaimana entity atau relationship diidentifikasi.

Mapping cardinality untuk relationship biner:

| Cardinality  | Makna                                                                         |
|--------------|-------------------------------------------------------------------------------|
| One-to-one   | Satu entity A berasosiasi paling banyak dengan satu entity B, dan sebaliknya. |
| One-to-many  | Satu entity A dapat berasosiasi dengan banyak entity B.                       |
| Many-to-one  | Banyak entity A dapat berasosiasi dengan satu entity B.                       |
| Many-to-many | Banyak entity A dapat berasosiasi dengan banyak entity B, dan sebaliknya.     |

Tabel 27: Mapping cardinality

**Total participation** berarti setiap entity dalam entity set wajib berpartisipasi dalam relationship. **Partial participation** berarti hanya sebagian entity yang berpartisipasi.

Keys dalam E-R:

- **Superkey**: atribut yang mengidentifikasi entity secara unik.
- **Candidate key**: superkey minimal.
- **Primary key**: candidate key yang dipilih sebagai identifikasi utama.
- **Key relationship set**: biasanya dibentuk dari gabungan primary key entity set yang berpartisipasi, ditambah pertimbangan cardinality.

Constraint E-R akan memengaruhi reduksi ke skema relasional. Misalnya many-to-many biasanya menjadi tabel baru, sedangkan many-to-one sering dapat direpresentasikan dengan foreign key pada sisi many.

### 10.2.4 Strong Entity, Weak Entity, dan Identifying Relationship

#### Inti Konsep.

Strong entity memiliki primary key sendiri, sedangkan weak entity tidak dapat diidentifikasi tanpa entity penopang. Konsep ini penting karena tidak semua objek dunia nyata memiliki identitas mandiri.

**Strong entity set** memiliki atribut yang cukup untuk membentuk primary key. **Weak entity set** tidak memiliki atribut cukup untuk membentuk primary key mandiri, sehingga bergantung pada identifying strong entity.

Komponen weak entity:

| Komponen                    | Peran                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------|
| Identifying strong entity   | Entity kuat yang memberi identitas pada weak entity.                                  |
| Identifying relationship    | Relationship yang menghubungkan weak entity dengan entity penopangnya.                |
| Partial key / discriminator | Atribut weak entity yang membedakan weak entity dalam satu entity penopang yang sama. |

Tabel 28: Komponen weak entity

Primary key weak entity dibentuk dari primary key strong entity penopang ditambah discriminator weak entity. Saat direduksi ke relasional, tabel weak entity memuat atribut lokalnya serta primary key dari strong entity sebagai foreign key sekaligus bagian primary key.

### 10.2.5 E-R Diagram dan UML Class Diagram

#### Inti Konsep.

E-R diagram adalah representasi visual entity, relationship, attribute, key, dan constraint. UML class diagram dapat digunakan sebagai alternatif visual untuk pemodelan data, terutama ketika sistem juga dimodelkan secara object-oriented.

Notasi E-R:

| Notasi          | Makna                          |
|-----------------|--------------------------------|
| Rectangle       | Entity set.                    |
| Diamond         | Relationship set.              |
| Oval            | Attribute.                     |
| Underline solid | Primary key.                   |
| Double oval     | Multivalued attribute.         |
| Dashed oval     | Derived attribute.             |
| Double line     | Total participation.           |
| Double diamond  | Identifying relationship.      |
| ISA triangle    | Specialization/generalization. |

Tabel 29: Notasi dasar E-R diagram

UML class diagram merepresentasikan entity sebagai class dengan atribut dan methods di dalam kotak bertingkat. Visibility dapat ditandai dengan +, -, dan #. Relationship digambar dengan garis dan diberi multiplicity seperti 0..1 atau 0..\*.

### 10.2.6 Specialization, Generalization, dan Inheritance

#### Inti Konsep.

Specialization memecah entity set umum menjadi subkelas yang lebih spesifik, sedangkan generalization menggabungkan beberapa entity set serupa menjadi superclass. Inheritance membuat subkelas mewarisi atribut dan relationship superclass.

Motivasi fitur ini adalah beberapa entity memiliki atribut umum sekaligus atribut khusus. Contoh: **Person** dapat dispesialisasi menjadi **Student** dan **Employee**. Keduanya mewarisi atribut umum seperti nama, tetapi memiliki atribut khusus seperti total SKS atau salary.

| Konsep                 | Makna                                                       |
|------------------------|-------------------------------------------------------------|
| Specialization         | Top-down: superclass dipecah menjadi subclass.              |
| Generalization         | Bottom-up: beberapa entity set digabung menjadi superclass. |
| Inheritance            | Subclass mewarisi atribut dan relationship superclass.      |
| Disjoint               | Satu entity hanya boleh berada pada satu subclass.          |
| Overlapping            | Satu entity dapat berada pada beberapa subclass.            |
| Total specialization   | Setiap entity superclass harus masuk minimal satu subclass. |
| Partial specialization | Entity boleh hanya berada di superclass.                    |

Tabel 30: Fitur specialization dan generalization

Saat direduksi ke relasional, salah satu metode adalah membuat tabel untuk superclass dan tabel untuk setiap subclass. Tabel subclass memuat primary key superclass ditambah atribut lokal subclass. Kelemahannya, untuk melihat data lengkap subclass, sering diperlukan join dengan tabel superclass.

### 10.2.7 Aggregation

#### Inti Konsep.

Aggregation adalah abstraksi yang memperlakukan relationship beserta entity pembentuknya sebagai entity tingkat tinggi. Konsep ini penting ketika kita perlu membuat relationship between relationships tanpa membuat relationship n-ary yang terlalu rumit.

Motivasi aggregation adalah ada situasi ketika suatu relationship sendiri perlu berpartisipasi dalam relationship lain. Contoh sumber: student dibimbing instructor pada project membentuk relationship `proj_guide`. Kombinasi ini kemudian dievaluasi oleh `evaluation`. Daripada membuat relationship besar yang mencampur semua entity, `proj_guide` diagregasi menjadi unit abstrak yang dapat dihubungkan ke `evaluation`.

Mekanismenya:

1. Identifikasi relationship yang secara konseptual menjadi objek pembahasan baru.
2. Bungkus relationship dan entity pembentuknya sebagai agregasi.
3. Hubungkan agregasi tersebut ke entity atau relationship lain.
4. Saat reduksi relasional, buat skema yang memuat key relationship agregat dan key entity terkait.

Aggregation membantu menjaga model tetap terbaca ketika hubungan antar fakta menjadi bertingkat.

### 10.2.8 Design Issues dalam E-R Model

#### Inti Konsep.

Design issues adalah keputusan pemodelan yang menentukan apakah suatu konsep sebaiknya menjadi entity, attribute, relationship, binary relationship, atau ternary relationship. Keputusan ini penting karena pemodelan yang salah dapat menciptakan redundansi dan masalah normalisasi.

Isu utama:

| Isu                    | Pertanyaan Desain                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Entity vs attribute    | Apakah suatu konsep cukup menjadi atribut, atau perlu menjadi entity tersendiri karena memiliki atribut/relasi sendiri? |
| Entity vs relationship | Apakah suatu kejadian adalah hubungan antar entity, atau objek yang perlu diidentifikasi sebagai entity?                |
| Binary vs ternary      | Apakah relasi tiga arah dapat dipecah menjadi relasi biner tanpa kehilangan makna?                                      |

Tabel 31: Isu desain E-R

Contoh sumber: menaruh `department_name` sebagai atribut murni pada `student` dapat buruk jika sebenarnya department adalah entity yang memiliki atribut dan hubungan sendiri. Lebih baik membuat relationship `stud_dept`.

Isu desain ini berhubungan langsung dengan normalisasi. Jika konsep yang seharusnya entity hanya dijadikan atribut, FD non-key dapat muncul dan membuat tabel hasil reduksi melanggar BCNF atau 3NF.

### 10.2.9 Reduksi E-R ke Skema Relasional

#### Inti Konsep.

Reduksi E-R ke skema relasional adalah proses menerjemahkan entity, relationship, attribute, dan constraint menjadi tabel, key, dan foreign key. Konsep ini penting karena E-R model adalah desain konseptual, sedangkan DBMS relasional membutuhkan skema tabel.

Aturan umum:

| Komponen E-R              | Reduksi ke Relasional                                                                                              |
|---------------------------|--------------------------------------------------------------------------------------------------------------------|
| Strong entity set         | Menjadi tabel dengan atribut entity; primary key sama dengan primary key entity.                                   |
| Weak entity set           | Menjadi tabel berisi atribut lokal, discriminator, dan primary key strong entity penopang.                         |
| Many-to-many relationship | Menjadi tabel baru berisi gabungan primary key entity partisipan dan atribut deskriptif relationship.              |
| Multivalued attribute     | Menjadi tabel baru berisi primary key entity asal dan atribut multivalue.                                          |
| Specialization            | Salah satu metode: tabel superclass ditambah tabel masing-masing subclass dengan key superclass dan atribut lokal. |
| Aggregation               | Direduksi dengan key relationship agregat, key entity terkait, dan atribut deskriptif tambahan.                    |

Tabel 32: Aturan reduksi E-R ke relasional

Pseudocode reduksi sederhana:

Listing 18: Pseudocode reduksi E-R ke skema relasional

```

1 Input: E-R diagram
2 Output: relational schemas
3
4 for each strong entity set E do
5     create relation E with all simple attributes
6     set primary key from E
7 for each weak entity set W do
8     create relation W with local attributes
9     add primary key of identifying strong entity
10    primary key := owner key union discriminator
11 for each many-to-many relationship R do
12    create relation R
13    add primary keys of participating entity sets

```

```

14 add descriptive attributes of R
15 for each multivalued attribute A of entity E do
16 create relation (E_key, A)

```

### 10.2.10 Hubungan E-R Model dengan Normalisasi

#### Inti Konsep.

E-R model yang dirancang dengan baik biasanya menghasilkan skema relasional yang tidak membutuhkan banyak normalisasi tambahan. Sebaliknya, E-R model yang buruk dapat menghasilkan FD non-key dan anomali pada tabel.

Contoh sumber: jika entity **employee** menyimpan **department\_name** dan **building** sebagai atribut biasa, muncul FD  $\text{department\_name} \rightarrow \text{building}$ . Jika **department\_name** bukan key **employee**, maka tabel hasilnya berpotensi melanggar BCNF/3NF.

Mekanisme masalah:

1. Perancang gagal mengenali **department** sebagai entity tersendiri.
2. Atribut **department** ditumpuk dalam entity **employee**.
3. FD antar atribut non-key muncul.
4. Tabel hasil reduksi mengandung redundansi.
5. Normalisasi perlu memecah tabel tersebut.

Jadi, E-R design dan relational database design saling melengkapi. E-R design mencegah kesalahan konseptual; normalisasi memeriksa kualitas skema relasional secara formal.

## 10.3 Algoritma dan Rumus Penting

### 10.3.1 Key Weak Entity

#### Makna Rumus.

Primary key weak entity dibentuk dari key strong entity penopang dan partial key weak entity. Rumus ini digunakan saat mereduksi weak entity ke tabel relasional.

$$PK(W) = PK(E_{owner}) \cup Discriminator(W) \quad (18)$$

dengan:

- $W$ : weak entity set.
- $E_{owner}$ : identifying strong entity set.
- $Discriminator(W)$ : partial key weak entity.

### 10.3.2 Skema Relationship Set Many-to-Many

#### Makna Rumus.

Relationship many-to-many direduksi menjadi tabel yang memuat key semua entity partisipan dan atribut deskriptif relationship.

$$R_{rel} = PK(E_1) \cup PK(E_2) \cup \dots \cup PK(E_n) \cup Attr(R) \quad (19)$$

dengan:

- $R_{rel}$ : skema relasi hasil reduksi relationship set.
- $E_i$ : entity set yang berpartisipasi.
- $Attr(R)$ : atribut deskriptif pada relationship.



## 11 Relational Database Design

Relational Database Design membahas cara menilai dan memperbaiki kualitas skema relasional setelah kebutuhan data diterjemahkan menjadi tabel. Fokus utamanya adalah mengenali redundansi, anomali pembaruan, dan ketergantungan antaratribut, lalu memakai teori *functional dependency* untuk mendekomposisi relasi ke bentuk yang lebih baik tanpa kehilangan informasi penting.

Materi ini berada di antara pemodelan E-R dan implementasi SQL. E-R model memberi rancangan konseptual, relational model memberi bentuk tabel, sedangkan relational database design memeriksa apakah tabel tersebut sudah layak dipakai dalam DBMS relasional.

### 11.1 Outline Fundamental Konsep Materi

1. **WAJIB** Tujuan desain relasional
  - (a) **WAJIB** Redundansi, anomali, dan kebutuhan nilai NULL
  - (b) **WAJIB** Relasi universal
2. **WAJIB** Functional dependency
  - (a) **WAJIB** Definisi formal  $\alpha \rightarrow \beta$
  - (b) **WAJIB** Trivial, non-trivial, dan transitive dependency
  - (c) **WAJIB** Superkey dan candidate key
  - (d) **PAHAM** Prime attribute
3. **WAJIB** Closure dan inferensi dependensi
  - (a) **WAJIB** Closure of functional dependencies  $F^+$
  - (b) **WAJIB** Attribute closure  $X^+$
  - (c) **WAJIB** Armstrong's axioms
  - (d) **PAHAM** Aturan turunan Armstrong
4. **WAJIB** Canonical cover
  - (a) **WAJIB** Extraneous attribute
  - (b) **WAJIB** Minimalisasi himpunan FD
  - (c) **WAJIB** Peran canonical cover dalam sintesis 3NF
5. **WAJIB** Decomposition
  - (a) **WAJIB** Lossless join
  - (b) **WAJIB** Dependency preservation
  - (c) **PAHAM** Uji lossless join untuk dekomposisi biner
6. **WAJIB** Boyce-Codd Normal Form (BCNF)
  - (a) **WAJIB** Definisi BCNF
  - (b) **WAJIB** Pengujian pelanggaran BCNF
  - (c) **WAJIB** Algoritma dekomposisi BCNF
7. **WAJIB** Third Normal Form (3NF)
  - (a) **WAJIB** Definisi 3NF
  - (b) **WAJIB** Algoritma sintesis 3NF

- (c) **WAJIB** Trade-off BCNF dan 3NF
- 8. **PAHAM** Bentuk normal lanjutan
  - (a) **PAHAM** First Normal Form (1NF)
  - (b) **PAHAM** Second Normal Form (2NF)
  - (c) **PAHAM** Multivalued dependency
  - (d) **PAHAM** Fourth Normal Form (4NF)
- 9. **PAHAM** Isu desain praktis
  - (a) **PAHAM** Denormalization for performance
  - (b) **PAHAM** Temporal data modeling
  - (c) **PAHAM** Batasan penegakan FD kompleks di SQL

## 11.2 Penjelasan Konsep

### 11.2.1 Tujuan Desain Relasional

#### Inti Konsep.

Tujuan desain relasional adalah menghasilkan sekumpulan skema relasi yang dapat menyimpan informasi tanpa redundansi yang tidak perlu dan tetap mudah digunakan untuk mengambil informasi. Konsep ini penting karena skema tabel yang buruk membuat data berulang, operasi pembaruan rawan inkonsisten, dan beberapa fakta sulit direpresentasikan tanpa nilai NULL.

Motivasi utamanya muncul dari masalah rancangan awal yang terlalu besar. Pada tahap awal, seluruh atribut yang menarik bagi sistem dapat saja dikumpulkan ke dalam satu **relasi universal**, yaitu satu skema besar yang memuat semua atribut sekaligus. Bentuk ini terlihat sederhana karena semua informasi berada di satu tempat, tetapi justru mencampur fakta dari beberapa entitas berbeda dalam satu tabel.

Definisi kerja desain relasional yang baik adalah desain yang menempatkan setiap fakta pada relasi yang sesuai dengan ketergantungan logisnya. Jika fakta tentang departemen disimpan berulang pada setiap baris instruktur, maka basis data menyimpan informasi yang sama berkali-kali. Pengulangan ini menimbulkan tiga anomali utama.

| Anomali                  | Makna dan Dampak                                                                                                                                                                             |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Update anomaly</b>    | Perubahan satu fakta harus dilakukan di banyak baris. Jika sebagian baris lupa diperbarui, basis data menjadi inkonsisten.                                                                   |
| <b>Insertion anomaly</b> | Fakta tertentu tidak dapat dimasukkan sebelum fakta lain tersedia. Misalnya departemen baru sulit dicatat jika belum ada instruktur pada departemen itu.                                     |
| <b>Deletion anomaly</b>  | Penghapusan satu tuple dapat menghapus fakta lain yang masih dibutuhkan. Misalnya menghapus instruktur terakhir pada departemen juga menghilangkan informasi gedung dan anggaran departemen. |

Tabel 33: Anomali pada rancangan relasional yang buruk

Sumber menggunakan contoh skema (*ID, name, salary, dept\_name, building, budget*). Jika beberapa instruktur bekerja pada departemen *Physics*, maka fakta bahwa *Physics* berada di gedung *Watson* dengan anggaran tertentu akan diulang pada setiap tuple instruktur. Inilah

contoh **repetition of information**, yaitu pengulangan fakta yang seharusnya disimpan satu kali.

Mekanisme umum memperbaiki rancangan adalah normalisasi:

1. Mulai dari skema relasional awal, baik hasil reduksi E-R, relasi universal, maupun rancangan ad hoc.
2. Identifikasi *functional dependencies* yang benar berdasarkan semantik dunia nyata.
3. Uji apakah skema berada dalam bentuk normal yang diinginkan.
4. Jika tidak, dekomposisi skema menjadi relasi lebih kecil.
5. Pastikan dekomposisi tidak menghilangkan informasi dan, jika mungkin, tetap mempertahankan dependensi.

Konsep ini berhubungan langsung dengan seluruh materi berikutnya: *functional dependency* memberi bahasa formal untuk menyatakan aturan data, sedangkan BCNF dan 3NF memberi kriteria apakah aturan tersebut sudah ditempatkan pada tabel yang tepat.

### 11.2.2 Functional Dependency

#### Inti Konsep.

*Functional dependency* (FD) adalah batasan logis antaratribut yang menyatakan bahwa nilai satu himpunan atribut menentukan nilai himpunan atribut lain. FD digunakan untuk mendeteksi redundansi, mendefinisikan key, dan menguji bentuk normal seperti BCNF dan 3NF.

Motivasi FD adalah kebutuhan untuk menuliskan aturan dunia nyata secara formal. Dalam relasi departemen, misalnya, nama departemen dapat menentukan gedung dan anggaran. Jika dua tuple memiliki **dept\_name** yang sama tetapi **building** berbeda, maka salah satu fakta pasti tidak konsisten dengan aturan tersebut.

Secara formal, untuk skema relasi  $R$ , functional dependency  $\alpha \rightarrow \beta$  berlaku pada  $R$  jika untuk setiap instans legal  $r(R)$  dan untuk setiap dua tuple  $t_1, t_2 \in r$ :

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta] \quad (20)$$

dengan  $\alpha$  dan  $\beta$  adalah himpunan atribut pada  $R$ . Artinya, kesamaan nilai pada atribut  $\alpha$  memaksa kesamaan nilai pada atribut  $\beta$ .

| Istilah                      | Definisi atau Peran                                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trivial FD</b>            | FD $\alpha \rightarrow \beta$ dengan $\beta \subseteq \alpha$ . Contoh: $A, B \rightarrow A$ . FD ini selalu benar dan tidak memberi batasan baru. |
| <b>Non-trivial FD</b>        | FD yang sisi kanannya tidak sepenuhnya berada di sisi kiri. FD jenis ini dapat mengindikasikan redundansi dan dipakai dalam pengujian BCNF.        |
| <b>Transitive dependency</b> | Dependensi tidak langsung: jika $\alpha \rightarrow \beta$ dan $\beta \rightarrow \gamma$ , maka $\alpha \rightarrow \gamma$ .                     |
| <b>Superkey</b>              | Himpunan atribut $K$ yang menentukan semua atribut dalam $R$ , yaitu $K \rightarrow R$ .                                                           |
| <b>Candidate key</b>         | Superkey minimal: $K \rightarrow R$ , tetapi tidak ada subset sejati $K$ yang juga menentukan $R$ .                                                |

Tabel 34: Istilah dasar dalam functional dependency

### Komentar 11.1: Catatan: Sumber Pengetahuan Umum

**Prime attribute** adalah atribut yang menjadi anggota setidaknya satu candidate key. Istilah ini penting pada 3NF karena 3NF mengizinkan pelanggaran BCNF tertentu apabila atribut di sisi kanan FD adalah prime attribute. Informasi ini melengkapi formulasi standar yang biasa muncul pada pembahasan 3NF di buku sistem basis data.

FD bekerja sebagai alat uji. Ketika tuple baru dimasukkan atau diperbarui, DBMS atau perancang harus memastikan bahwa semua FD yang berlaku tetap benar. Dalam praktik SQL, FD yang murah ditegakkan biasanya adalah FD berbasis **primary key** atau **unique**; FD yang tidak berbasis key lebih sulit ditegakkan secara langsung.

Hubungannya dengan normalisasi sangat kuat: BCNF mensyaratkan determinan FD non-trivial harus superkey, sedangkan 3NF memberi relaksasi tertentu. Tanpa FD, tidak ada dasar formal untuk mengatakan bahwa suatu tabel perlu dipecah.

### 11.2.3 Closure dan Inferensi Dependensi

#### Inti Konsep.

Closure adalah cara menghitung semua dependensi yang secara logis mengikuti dari himpunan FD awal.  $F^+$  menyatakan semua FD yang terimplikasi oleh  $F$ , sedangkan  $X^+$  menyatakan semua atribut yang dapat ditentukan oleh atribut  $X$ .

Motivasinya adalah bahwa aturan eksplisit sering menghasilkan aturan implisit. Jika  $NIM \rightarrow PRODI$  dan  $PRODI \rightarrow FAKULTAS$ , maka  $NIM \rightarrow FAKULTAS$  benar walaupun tidak ditulis langsung. Desain relasional harus memperhitungkan implikasi semacam ini karena pelanggaran normal form dapat berasal dari dependensi turunan.

**Closure of functional dependencies**  $F^+$  adalah himpunan seluruh FD yang dapat diturunkan secara logis dari  $F$ . Menghitung  $F^+$  secara penuh dapat mahal karena jumlah FD yang mungkin sangat besar. Karena itu, perancang sering memakai **attribute closure**  $X^+$ , yaitu himpunan semua atribut yang dapat ditentukan oleh  $X$  berdasarkan  $F$ .

Aksioma Armstrong menyediakan aturan inferensi dasar yang *sound* dan *complete*. *Sound* berarti aturan tidak menghasilkan FD palsu; *complete* berarti semua FD yang benar-benar terimplikasi dapat diturunkan dengan aturan tersebut.

| Aksioma      | Bentuk                                                                                              | Makna                                                                  |
|--------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Reflexivity  | Jika $\beta \subseteq \alpha$ , maka $\alpha \rightarrow \beta$ .                                   | Himpunan atribut selalu menentukan subset-nya sendiri.                 |
| Augmentation | Jika $\alpha \rightarrow \beta$ , maka $\gamma\alpha \rightarrow \gamma\beta$ .                     | Menambahkan atribut yang sama pada kedua sisi FD menjaga kebenaran FD. |
| Transitivity | Jika $\alpha \rightarrow \beta$ dan $\beta \rightarrow \gamma$ , maka $\alpha \rightarrow \gamma$ . | Dependensi dapat dirantai secara logis.                                |

Tabel 35: Aksioma Armstrong

### Komentar 11.2: Catatan: Sumber Pengetahuan Umum

Aturan turunan yang lazim dipakai adalah *union*, *decomposition*, dan *pseudotransitivity*. Jika  $\alpha \rightarrow \beta$  dan  $\alpha \rightarrow \gamma$ , maka  $\alpha \rightarrow \beta\gamma$  (*union*). Jika  $\alpha \rightarrow \beta\gamma$ , maka  $\alpha \rightarrow \beta$  dan  $\alpha \rightarrow \gamma$  (*decomposition*). Jika  $\alpha \rightarrow \beta$  dan  $\gamma\beta \rightarrow \delta$ , maka  $\alpha\gamma \rightarrow \delta$  (*pseudotransitivity*).

Algoritma attribute closure:

Listing 19: Pseudocode attribute closure  $X^+$

```
1 Input: attribute set X, functional dependencies F
```

```

2 Output: X_plus
3
4 X_plus := X
5 repeat
6     changed := false
7     for each FD Y -> Z in F do
8         if Y subset of X_plus and Z not subset of X_plus then
9             X_plus := X_plus union Z
10            changed := true
11 until changed = false
12 return X_plus

```

Untuk menguji apakah  $X$  adalah superkey bagi  $R$ , hitung  $X^+$ . Jika  $X^+$  memuat semua atribut dalam  $R$ , maka  $X \rightarrow R$  dan  $X$  adalah superkey. Perhitungan ini dipakai berulang pada pengujian BCNF, 3NF, canonical cover, dan dependency preservation.

#### 11.2.4 Canonical Cover

##### Inti Konsep.

*Canonical cover* adalah himpunan FD yang ekuivalen dengan himpunan FD semula, tetapi sudah disederhanakan dengan menghapus atribut dan dependensi yang tidak perlu. Ia penting karena menjadi input utama algoritma sintesis 3NF dan mengurangi biaya pengecekan constraint.

Motivasi canonical cover adalah efisiensi dan kebersihan logika. Dua himpunan FD  $F$  dan  $G$  ekuivalen jika  $F^+ = G^+$ . Jika  $G$  jauh lebih kecil tetapi closure-nya sama, maka  $G$  lebih mudah dipakai untuk desain, pengujian, dan sintesis skema.

**Extraneous attribute** adalah atribut pada sisi kiri atau kanan FD yang dapat dihapus tanpa mengubah closure himpunan FD. Atribut dapat ekstranius pada **left-hand side** atau pada **right-hand side**.

| Kasus                                                               | Cara Uji                                                                                                                       |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Atribut $A \in \alpha$ pada sisi kiri<br>$\alpha \rightarrow \beta$ | Hitung $(\alpha - \{A\})^+$ terhadap $F$ . Jika hasilnya tetap memuat $\beta$ , maka $A$ ekstranius.                           |
| Atribut $A \in \beta$ pada sisi<br>kanan $\alpha \rightarrow \beta$ | Ganti FD menjadi $\alpha \rightarrow (\beta - \{A\})$ , lalu cek apakah $\alpha^+$ tetap memuat $A$ . Jika ya, $A$ ekstranius. |

Tabel 36: Uji atribut ekstranius

Pseudocode canonical cover:

Listing 20: Pseudocode canonical cover

```

1 Input: functional dependencies F
2 Output: canonical cover Fc
3
4 Fc := F
5 repeat
6     combine dependencies in Fc with the same left-hand side
7     find an FD alpha -> beta in Fc with an extraneous attribute
8     if an extraneous attribute exists then
9         delete that attribute from the FD
10    if an FD has empty right-hand side then
11        delete that FD
12 until Fc no longer changes
13 return Fc

```

Sumber memberi contoh relasi `cust_banker_branch` dengan FD:

$customer\_id, employee\_id \rightarrow branch\_name, type$

$$\begin{aligned} & employee\_id \rightarrow branch\_name \\ & customer\_id, branch\_name \rightarrow employee\_id \end{aligned}$$

Atribut **branch\_name** pada sisi kanan FD pertama ekstranius, sehingga canonical cover menjadi:

$$\begin{aligned} & customer\_id, employee\_id \rightarrow type \\ & employee\_id \rightarrow branch\_name \\ & customer\_id, branch\_name \rightarrow employee\_id \end{aligned}$$

Canonical cover menghubungkan teori FD dengan algoritma desain. Tanpa canonical cover, sintesis 3NF dapat menghasilkan skema berlebih atau membawa dependensi yang sebenarnya redundan.

### 11.2.5 Decomposition, Lossless Join, dan Dependency Preservation

#### Inti Konsep.

Dekomposisi adalah pemecahan satu skema relasi besar menjadi beberapa skema yang lebih kecil. Dekomposisi yang baik harus *lossless join*, sehingga informasi tidak hilang atau bertambah palsu saat di-join kembali, dan sebaiknya *dependency preserving*, sehingga FD tetap dapat dicek tanpa join mahal.

Motivasinya adalah memperbaiki relasi yang melanggar bentuk normal. Namun memecah tabel tidak boleh dilakukan sembarangan. Jika tabel dipecah berdasarkan atribut yang salah, natural join tabel pecahan dapat menghasilkan tuple palsu atau kehilangan kemampuan memeriksa aturan data.

Misalkan skema  $R$  didekomposisi menjadi  $R_1, R_2, \dots, R_n$ . Dekomposisi bersifat **lossless** jika untuk setiap instans legal  $r(R)$ , join proyeksi relasi hasil dekomposisi menghasilkan kembali  $r$  secara tepat:

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \dots \bowtie \Pi_{R_n}(r) \quad (21)$$

#### Komentar 11.3: Catatan: Sumber Pengetahuan Umum

Untuk dekomposisi biner  $R$  menjadi  $R_1$  dan  $R_2$ , uji lossless yang umum dipakai adalah:  $(R_1 \cap R_2) \rightarrow R_1$  atau  $(R_1 \cap R_2) \rightarrow R_2$  harus berada dalam  $F^+$ . Dengan kata lain, atribut irisan harus menjadi superkey untuk salah satu relasi hasil dekomposisi.

**Dependency preservation** berarti seluruh FD dalam  $F$  dapat ditegakkan dengan memeriksa tabel hasil dekomposisi secara individual. Jika suatu FD hanya dapat dicek setelah melakukan join, maka dekomposisi tidak dependency preserving. Ini penting karena join untuk memvalidasi setiap update mahal dan tidak praktis.

Contoh dependency preservation: untuk  $R(A, B, C)$  dengan  $F = \{A \rightarrow B, B \rightarrow C\}$ , dekomposisi menjadi  $R_1(A, B)$  dan  $R_2(B, C)$  mempertahankan kedua FD. Sebaliknya, dekomposisi menjadi  $R_1(A, B)$  dan  $R_2(A, C)$  tidak mempertahankan  $B \rightarrow C$  tanpa join.

Pseudocode uji sederhana preservation untuk satu FD  $\alpha \rightarrow \beta$ :

Listing 21: Pseudocode uji dependency preservation

```

1 Input: FD alpha -> beta, decomposition R1..Rn, dependencies F
2 Output: preserved or not preserved
3
4 result := alpha
5 repeat
6     old_result := result
7     for each schema Ri in decomposition do
8         result := result union ((result intersect Ri)+ intersect Ri)
9 until result = old_result

```

```

10
11 if beta subset of result then
12     return preserved
13 else
14     return not_preserved

```

Dekomposisi adalah jembatan menuju BCNF dan 3NF. BCNF mengutamakan penghilangan redundansi berbasis FD, sedangkan 3NF memberi kompromi agar dependency preservation tetap dapat dijamin.

### 11.2.6 Boyce-Codd Normal Form (BCNF)

#### Inti Konsep.

BCNF adalah bentuk normal yang menuntut setiap FD non-trivial memiliki determinant berupa superkey. Ia penting karena menghilangkan redundansi yang dapat dijelaskan oleh functional dependency.

Motivasi BCNF adalah memperketat desain agar setiap fakta ditempatkan pada relasi yang determinant-nya benar-benar mengidentifikasi tuple. Jika ada FD  $\alpha \rightarrow \beta$  dan  $\alpha$  bukan superkey, maka nilai  $\alpha$  dapat muncul berulang dan membawa nilai  $\beta$  yang sama berulang pula.

Secara formal, skema relasi  $R$  berada dalam BCNF terhadap  $F$  jika untuk setiap FD  $\alpha \rightarrow \beta$  dalam  $F^+$ , salah satu kondisi berikut benar:

1.  $\alpha \rightarrow \beta$  trivial, yaitu  $\beta \subseteq \alpha$ .
2.  $\alpha$  adalah superkey untuk  $R$ .

Mekanisme pengujian BCNF:

1. Ambil FD non-trivial  $\alpha \rightarrow \beta$  yang berlaku pada relasi.
2. Hitung  $\alpha^+$  terhadap FD yang relevan.
3. Jika  $\alpha^+$  memuat semua atribut  $R$ , maka  $\alpha$  superkey dan FD tidak melanggar BCNF.
4. Jika  $\alpha^+$  tidak memuat seluruh atribut  $R$ , maka FD tersebut melanggar BCNF.

Algoritma dekomposisi BCNF:

Listing 22: Pseudocode dekomposisi BCNF

```

1 Input: relation schema R, functional dependencies F
2 Output: BCNF decomposition
3
4 result := {R}
5 while exists schema Ri in result that violates BCNF do
6     choose a non-trivial FD alpha -> beta on Ri
7     such that alpha is not a superkey of Ri
8     replace Ri with:
9         R1 := alpha union beta
10        R2 := Ri - (beta - alpha)
11 return result

```

Sumber memberi contoh relasi `class`:

*course\_id, title, dept\_name, credits, sec\_id, semester, year,  
building, room\_number, capacity, time\_slot\_id*

FD  $\text{course\_id} \rightarrow \text{title, dept\_name, credits}$  melanggar BCNF karena `course_id` bukan superkey untuk keseluruhan `class`. Relasi dipecah menjadi `course(course_id, title, dept_name, credits)` dan relasi sisa. Pada relasi sisa, FD  $\text{building, room\_number} \rightarrow$

capacity juga melanggar BCNF, sehingga dipisah menjadi `classroom(building, room_number, capacity)` dan `section`.

BCNF berhubungan langsung dengan lossless join karena algoritma dekomposisinya dirancang menghasilkan dekomposisi lossless. Kelemahannya, BCNF tidak selalu dependency preserving.

### 11.2.7 Third Normal Form (3NF)

#### Inti Konsep.

3NF adalah relaksasi BCNF yang tetap mengurangi redundansi besar, tetapi dirancang agar selalu dapat diperoleh dekomposisi yang lossless dan dependency preserving. Ia penting ketika BCNF menghilangkan kemampuan memeriksa FD tanpa join.

Motivasi 3NF muncul dari trade-off desain. BCNF lebih ketat, tetapi ada kasus ketika semua relasi BCNF tidak dapat mempertahankan semua dependency. 3NF mengizinkan beberapa FD yang determinan-nya bukan superkey, asalkan atribut di sisi kanan merupakan bagian dari candidate key.

Relasi  $R$  berada pada 3NF terhadap  $F$  jika untuk setiap FD  $\alpha \rightarrow \beta$  dalam  $F^+$ , setidaknya satu syarat berikut berlaku:

1.  $\alpha \rightarrow \beta$  trivial.
2.  $\alpha$  adalah superkey untuk  $R$ .
3. Setiap atribut  $A \in \beta - \alpha$  adalah prime attribute.

Algoritma sintesis 3NF:

Listing 23: Pseudocode sintesis 3NF

```
1 Input: relation schema R, functional dependencies F
2 Output: 3NF decomposition
3
4 compute canonical cover Fc for F
5 result := empty set
6 for each FD alpha -> beta in Fc do
7     if no schema in result contains alpha union beta then
8         add schema (alpha union beta) to result
9 if no schema in result contains a candidate key for R then
10     add one schema containing a candidate key for R
11 remove every schema that is subset of another schema
12 return result
```

Sumber memakai contoh `cust_banker_branch(customer_id, employee_id, branch_name, type)`. Setelah canonical cover diperoleh, sintesis menghasilkan skema:

$(customer\_id, employee\_id, type)$

$(employee\_id, branch\_name)$

$(customer\_id, branch\_name, employee\_id)$

Karena salah satu skema sudah memuat candidate key, tidak perlu menambahkan skema key baru.

Perbandingan BCNF dan 3NF:



| Aspek                   | BCNF                                                    | 3NF                                                           |
|-------------------------|---------------------------------------------------------|---------------------------------------------------------------|
| Kekuatan normalisasi    | Lebih ketat; determinant FD non-trivial harus superkey. | Lebih longgar; mengizinkan sisi kanan berupa prime attribute. |
| Redundansi berbasis FD  | Sangat kecil karena pelanggaran FD dipecah.             | Masih mungkin ada redundansi tertentu.                        |
| Lossless join           | Dapat dijamin oleh algoritma dekomposisi.               | Dapat dijamin oleh algoritma sintesis.                        |
| Dependency preservation | Tidak selalu terjamin.                                  | Dapat dijamin.                                                |

Tabel 37: Trade-off BCNF dan 3NF

Dalam praktik SQL, BCNF sering lebih disukai jika constraint FD non-key sulit ditegakkan. SQL secara langsung mudah menegakkan FD melalui **primary key** dan **unique**; FD yang lebih kompleks dapat memerlukan mekanisme mahal seperti assertions atau trigger.

### 11.2.8 Bentuk Normal Lanjutan

#### Inti Konsep.

Bentuk normal lanjutan memperluas normalisasi di luar FD biasa. 1NF memastikan semua nilai atomik, 2NF menangani ketergantungan parsial pada candidate key komposit, sedangkan 4NF menangani multivalued dependency yang masih dapat menimbulkan redundansi walaupun relasi sudah BCNF.

Motivasi bentuk normal lanjutan adalah bahwa tidak semua masalah desain ditangkap oleh BCNF dan 3NF. Sebelum FD dianalisis, relasi harus memenuhi 1NF. Setelah BCNF, masih ada kasus redundansi yang bersumber dari *multivalued dependency*.

**First Normal Form (1NF)** mensyaratkan setiap domain atribut bersifat atomik. Nilai atomik adalah nilai yang tidak perlu dipecah lagi oleh model relasional. Misalnya satu nomor akun dapat dianggap atomik, tetapi satu sel berisi himpunan banyak nomor akun tidak atomik.

#### Komentar 11.4: Catatan: Sumber Pengetahuan Umum

**Second Normal Form (2NF)** adalah bentuk normal historis yang mensyaratkan relasi sudah 1NF dan setiap atribut non-prime bergantung penuh pada seluruh candidate key, bukan hanya pada sebagian candidate key komposit. Dalam praktik modern, 2NF sering dipelajari sebagai tahap antara menuju 3NF; jika relasi sudah 3NF, maka masalah 2NF juga sudah teratasi.

**Multivalued dependency (MVD)** menuliskan fakta bahwa satu atribut menentukan sekumpulan nilai atribut lain secara independen dari atribut lain. Notasinya:

$$\alpha \twoheadrightarrow \beta \quad (22)$$

dibaca  $\alpha$  multidetermines  $\beta$ . Sumber memberi contoh **isbn**  $\twoheadrightarrow$  **author**: satu ISBN dapat memiliki banyak author.

**Fourth Normal Form (4NF)** menggunakan MVD sebagai dasar. Relasi berada pada 4NF jika untuk setiap MVD non-trivial  $\alpha \twoheadrightarrow \beta$ ,  $\alpha$  adalah superkey. Jika tidak, relasi perlu didekomposisi lebih lanjut.

| Bentuk | Fokus Utama                                                               |
|--------|---------------------------------------------------------------------------|
| 1NF    | Domain atomik; tidak ada nilai multivalued atau composite dalam satu sel. |
| 2NF    | Menghindari ketergantungan parsial pada candidate key komposit.           |
| 3NF    | Mengurangi dependensi non-key sambil menjaga dependency preservation.     |
| BCNF   | Memaksa determinant FD non-trivial menjadi superkey.                      |
| 4NF    | Menghilangkan redundansi akibat multivalued dependency.                   |

Tabel 38: Fokus beberapa bentuk normal

Bentuk normal lanjutan berkaitan dengan BCNF karena BCNF belum selalu cukup. Jika redundansi berasal dari MVD, maka desain perlu dianalisis dengan 4NF, bukan hanya FD.

### 11.2.9 Denormalization for Performance

#### Inti Konsep.

Denormalization adalah keputusan sadar untuk menggabungkan kembali informasi yang sudah dinormalisasi demi mempercepat pembacaan data. Ia penting karena desain yang paling normal tidak selalu memberi performa query terbaik pada beban kerja tertentu.

Motivasi denormalisasi adalah trade-off antara kebersihan desain dan performa. Normalisasi memecah data agar redundansi berkurang, tetapi query tertentu menjadi membutuhkan banyak join. Jika query tersebut sangat sering dijalankan dan update relatif jarang, perancang dapat menyimpan data gabungan secara sengaja.

Mekanismenya:

1. Identifikasi query yang mahal karena terlalu banyak join.
2. Tentukan atribut dari beberapa relasi yang sering dibaca bersama.
3. Buat relasi denormalized atau materialized structure yang menyimpan hasil gabungan.
4. Pastikan ada strategi menjaga konsistensi saat data sumber berubah.
5. Evaluasi trade-off antara percepatan lookup dan biaya update.

Sumber memberi contoh tabel **course** dan **prereq**. Desain normal memisahkan informasi course dan prerequisite, sehingga query harus melakukan join. Denormalisasi dapat menyimpan atribut course dan prereq berdampingan agar lookup lebih cepat.

Biayanya adalah ruang penyimpanan ekstra dan operasi update yang lebih mahal. Jika judul course berubah, semua salinan pada struktur denormalized harus ikut diperbarui. Karena itu denormalisasi bukan pengganti normalisasi, melainkan keputusan performa setelah struktur normal dipahami.

Konsep ini terhubung dengan data warehouse. Model dimensional sering sengaja merelaksasi normalisasi, terutama pada star schema, agar query analitik read-only menjadi cepat.

### 11.2.10 Temporal Data Modeling

#### Inti Konsep.

Temporal data modeling adalah cara memodelkan data yang berubah sepanjang waktu dengan menyimpan riwayat validitas fakta. Ia penting karena update biasa hanya menyimpan snapshot terbaru dan dapat menghapus informasi historis yang dibutuhkan.

Motivasi temporal modeling adalah kebutuhan menyimpan data saat ini sekaligus data historis. Jika nama mata kuliah CS-201 Intro. to C berubah menjadi CS-201 Intro. to

Java, update biasa akan menghapus fakta lama. Dalam desain temporal, kedua versi dapat disimpan beserta interval waktu berlakunya.

Mekanismenya:

1. Tambahkan atribut waktu seperti **start** dan **end** pada tuple historis.
2. Definisikan periode validitas fakta dunia nyata.
3. Sesuaikan key agar mempertimbangkan waktu, misalnya menggunakan temporal primary key.
4. Modifikasi operasi join agar hanya menggabungkan tuple dengan interval waktu yang relevan atau saling tumpang tindih.
5. Gunakan predikat temporal seperti *overlaps*, *contains*, *before*, dan *after*.

| Istilah                               | Definisi atau Peran                                                                            |
|---------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Snapshot</b>                       | Keadaan data pada satu titik waktu tertentu.                                                   |
| <b>Temporal functional dependency</b> | FD yang berlaku pada titik atau interval waktu tertentu, bukan selalu sepanjang umur relasi.   |
| <b>Temporal primary key</b>           | Key yang memasukkan dimensi waktu agar beberapa versi historis objek yang sama dapat disimpan. |
| <b>Temporal join</b>                  | Join yang memperhitungkan tumpang tindih interval waktu antar tuple.                           |

Tabel 39: Istilah dalam temporal data modeling

Temporal modeling memperluas konsep FD, key, dan operasi relasional. Ia tetap bagian dari desain relasional karena perubahan struktur waktu dapat memengaruhi normal form dan cara constraint didefinisikan.

## 11.3 Algoritma dan Rumus Penting

### 11.3.1 Functional Dependency

#### Makna Rumus.

Rumus FD menyatakan kapan satu himpunan atribut menentukan himpunan atribut lain. Rumus ini digunakan untuk mendefinisikan key, menguji normal form, dan menentukan apakah dekomposisi diperlukan.

$$\alpha \rightarrow \beta \iff \forall t_1, t_2 \in r, t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta] \quad (23)$$

dengan:

- $R$ : skema relasi.
- $r$ : instans legal dari  $R$ .
- $\alpha, \beta$ : himpunan atribut pada  $R$ .
- $t_1, t_2$ : tuple dalam  $r$ .

### 11.3.2 Attribute Closure

#### Tujuan Algoritma.

Attribute closure menghitung semua atribut yang dapat ditentukan oleh suatu himpunan atribut. Hasilnya dipakai untuk menguji superkey, implied dependency, BCNF, 3NF, dan dependency preservation.

Listing 24: Pseudocode attribute closure

```
1 Input: X, F
2 Output: X_plus
3
4 X_plus := X
5 repeat
6     old := X_plus
7     for each dependency Y -> Z in F do
8         if Y subset of X_plus then
9             X_plus := X_plus union Z
10 until X_plus = old
11 return X_plus
```

### 11.3.3 BCNF Decomposition

#### Tujuan Algoritma.

Algoritma dekomposisi BCNF memecah relasi yang melanggar BCNF menjadi relasi lebih kecil yang lossless dan memenuhi BCNF. Algoritma ini tidak selalu mempertahankan semua dependency.

Listing 25: Pseudocode dekomposisi BCNF

```
1 Input: R, F
2 Output: result
3
4 result := {R}
5 while exists Ri in result and FD alpha -> beta violates BCNF on Ri do
6     result := result - {Ri}
7     result := result union {alpha union beta}
8     result := result union {Ri - (beta - alpha)}
9 return result
```

### 11.3.4 3NF Synthesis

#### Tujuan Algoritma.

Algoritma sintesis 3NF membuat dekomposisi yang berada pada 3NF, lossless, dan dependency preserving. Input pentingnya adalah canonical cover agar relasi hasil tidak membawa atribut atau dependensi berlebihan.

Listing 26: Pseudocode sintesis 3NF

```
1 Input: R, F
2 Output: schemas
3
4 Fc := canonical_cover(F)
5 schemas := {}
6 for each alpha -> beta in Fc do
7     if no schema in schemas contains alpha union beta then
8         schemas := schemas union {alpha union beta}
```

```
9 if no schema in schemas contains a candidate key of R then
10     schemas := schemas union {one candidate key of R}
11 remove schemas that are subsets of other schemas
12 return schemas
```

## 12 Integrity Constraints

Integrity Constraints membahas aturan yang menjaga konsistensi logis basis data ketika data disisipkan, dihapus, atau diperbarui. Materi ini adalah jembatan antara teori model relasional, desain skema, dan implementasi SQL: key, foreign key, domain, dan aturan bisnis tidak cukup hanya dipahami sebagai konsep, tetapi harus dinyatakan sebagai constraint yang ditegakkan oleh DBMS.

Fokus utama untuk ujian adalah memahami jenis constraint, kapan constraint diperiksa, apa akibat operasi modifikasi terhadap constraint, dan mengapa constraint deklaratif di SQL lebih aman daripada memencarkan aturan ke kode aplikasi.

### 12.1 Outline Fundamental Konsep Materi

1. **WAJIB** Tujuan integrity constraints
  - (a) **WAJIB** Konsistensi data
  - (b) **WAJIB** Integrity constraints vs security constraints
  - (c) **PAHAM** Batasan constraint: menjaga konsistensi, bukan menjamin kebenaran dunia nyata
2. **WAJIB** Pendekatan penegakan constraint
  - (a) **WAJIB** Prosedur penambahan constraint baru
  - (b) **WAJIB** Declarative integrity support
  - (c) **PAHAM** Procedural integrity support
3. **WAJIB** Konstrain fundamental model relasional
  - (a) **WAJIB** Domain constraints
  - (b) **WAJIB** Entity integrity
  - (c) **WAJIB** Referential integrity
4. **WAJIB** Implementasi constraint SQL
  - (a) **WAJIB** not null
  - (b) **WAJIB** primary key
  - (c) **WAJIB** unique
  - (d) **WAJIB** check
  - (e) **PAHAM** Named constraint
5. **WAJIB** Foreign key dan referential actions
  - (a) **WAJIB** foreign key references
  - (b) **WAJIB** restrict dan no action
  - (c) **WAJIB** cascade
  - (d) **WAJIB** set null dan set default
6. **PAHAM** Constraint pada transaksi dan logika bisnis
  - (a) **PAHAM** Transitory violation
  - (b) **PAHAM** Deferred constraint checking
  - (c) **PAHAM** Assertions
  - (d) **PAHAM** Triggers

- (e) **PAHAM** Functions dan procedures
- 7. **WAJIB** Hubungan integrity constraints dengan materi lain
  - (a) **WAJIB** Keys dan relational model
  - (b) **WAJIB** DDL SQL
  - (c) **WAJIB** Relational database design dan normalisasi

## 12.2 Penjelasan Konsep

### 12.2.1 Tujuan Integrity Constraints

#### Inti Konsep.

Integrity constraints adalah aturan logis yang memastikan perubahan data oleh pengguna yang sah tidak membuat basis data kehilangan konsistensi. Constraint penting karena DBMS harus menolak data yang melanggar struktur dan hubungan logis yang telah ditetapkan dalam skema.

Motivasi utama integrity constraints adalah melindungi basis data dari kerusakan yang tidak disengaja. Pengguna yang berwenang dapat saja memasukkan nilai yang salah tipe, menghapus baris induk yang masih dirujuk, atau memperbarui key menjadi nilai yang membuat tuple tidak dapat diidentifikasi. Constraint memberi batas formal agar operasi seperti **insert**, **delete**, dan **update** tidak merusak konsistensi.

Secara konseptual, **integrity** berarti konsistensi data di dalam basis data. Integrity constraint tidak menjamin kebenaran absolut fakta dunia nyata. Jika gaji yang dimasukkan adalah 50000, constraint dapat memastikan nilainya bertipe numerik dan berada di rentang sah, tetapi tidak selalu bisa membuktikan bahwa angka tersebut benar-benar gaji aktual pegawai.

| Jenis Batasan                | Fokus                                                                                                                                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Integrity constraints</b> | Mencegah kerusakan konsistensi data oleh operasi pengguna yang sah. Contoh: saldo harus melewati batas minimum, nomor telepon pelanggan tidak boleh kosong, foreign key harus merujuk tuple yang ada. |
| <b>Security constraints</b>  | Mencegah akses atau operasi oleh pengguna yang tidak berwenang. Contoh: hanya admin yang boleh menghapus data atau hanya dosen tertentu yang boleh mengubah nilai.                                    |

Tabel 40: Perbedaan integrity constraints dan security constraints

Contoh dari sumber mencakup aturan seperti saldo akun cek harus lebih besar dari nilai tertentu, gaji karyawan bank minimal bernilai tertentu, dan pelanggan harus memiliki nomor telepon non-null. Ketiga contoh tersebut menunjukkan bahwa constraint bukan hanya sintaks SQL, tetapi pernyataan aturan organisasi.

Konsep ini berhubungan dengan relational model karena constraint mendefinisikan instans relasi mana yang legal. Ia juga berhubungan dengan SQL karena constraint biasanya ditulis dalam DDL agar ditegakkan otomatis oleh DBMS.

### 12.2.2 Pendekatan Penegakan Constraint

#### Inti Konsep.

Constraint dapat ditegakkan secara deklaratif oleh DBMS atau secara prosedural oleh program aplikasi. Pendekatan deklaratif lebih kuat karena aturan ditulis pada skema dan otomatis berlaku untuk semua operasi data.

Motivasi membedakan pendekatan ini adalah menjaga aturan tetap terpusat. Jika aturan bisnis hanya ditulis di aplikasi, aplikasi lain atau query manual dapat melewati aturan tersebut. Jika constraint dideklarasikan di DBMS, semua jalur modifikasi data harus mematuhi.

Ketika constraint baru ditambahkan ke basis data yang sudah berisi data, DBMS tidak langsung menerimanya begitu saja. Mekanismenya:

1. DBMS mengevaluasi kondisi basis data saat ini terhadap constraint baru.
2. Jika ada data lama yang melanggar, constraint ditolak.
3. Jika semua data lama memenuhi, constraint diterima.
4. Sejak saat itu, setiap operasi modifikasi harus mematuhi constraint tersebut.

| Pendekatan                           | Definisi dan Implikasi                                                                                                                                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Declarative integrity support</b> | Constraint dideklarasikan langsung di DBMS, biasanya melalui DDL seperti <code>create table</code> atau <code>alter table</code> . DBMS otomatis menolak operasi yang melanggar. |
| <b>Procedural integrity support</b>  | Aturan ditulis dalam program aplikasi. Semua aplikasi yang mengakses database harus mengulang logika yang sama; jika ada satu aplikasi lupa, konsistensi dapat rusak.            |

Tabel 41: Pendekatan penegakan integrity constraints

Pendekatan deklaratif terkait langsung dengan DDL SQL. Aturan seperti `primary key`, `foreign key`, dan `check` seharusnya berada di skema, bukan hanya di kode aplikasi.

### 12.2.3 Domain Constraints

#### Inti Konsep.

Domain constraint membatasi nilai atribut agar berasal dari domain yang sah. Constraint ini penting karena menjadi lapisan paling dasar untuk memastikan nilai yang masuk ke kolom memiliki tipe, ukuran, dan rentang yang sesuai.

Motivasinya sederhana: operasi relasional hanya bermakna jika nilai atribut berasal dari domain yang benar. Atribut `salary` tidak seharusnya berisi teks bebas, atribut tanggal tidak seharusnya berisi angka acak, dan atribut semester hanya boleh memuat nilai semester yang dikenal.

Secara formal, untuk atribut  $A$  dengan domain  $D_A$ , setiap tuple  $t$  dalam relasi legal harus memenuhi:

$$t[A] \in D_A \quad (24)$$

Domain biasanya memuat nama domain, makna, tipe data, ukuran atau panjang, dan nilai atau rentang nilai yang diizinkan. Di SQL, domain constraint tampak pada tipe data seperti `integer`, `varchar(20)`, `numeric(12,2)`, serta pembatas tambahan seperti `check`.

Mekanisme pemeriksaannya:

1. Saat nilai baru masuk, DBMS membaca domain atribut tujuan.



2. DBMS memeriksa apakah nilai sesuai tipe dan batas domain.
3. Jika sesuai, operasi boleh berlanjut ke pemeriksaan constraint lain.
4. Jika tidak sesuai, operasi ditolak.

Domain constraint menjadi dasar untuk **not null**, **check**, dan validasi tipe data. Tanpa domain yang benar, constraint tingkat key dan foreign key pun tidak cukup untuk menjaga kualitas data.

#### 12.2.4 Entity Integrity

##### Inti Konsep.

Entity integrity memastikan setiap relasi memiliki primary key yang dapat mengidentifikasi tuple secara unik dan tidak bernilai null. Konsep ini penting karena tuple yang tidak dapat diidentifikasi tidak dapat dirujuk, diperbarui, atau dibedakan secara aman.

Motivasi entity integrity adalah kebutuhan identitas. Dalam tabel relasional, setiap baris merepresentasikan fakta atau objek tertentu. Jika key bernilai null, DBMS tidak dapat memastikan tuple mana yang dimaksud. Jika dua tuple memiliki primary key sama, keduanya tidak dapat dibedakan sebagai entitas berbeda.

Jika  $K$  adalah primary key relasi  $R$ , maka setiap instans legal  $r(R)$  harus memenuhi:

$$\forall t_1, t_2 \in r, t_1[K] = t_2[K] \Rightarrow t_1 = t_2 \quad (25)$$

dan:

$$\forall t \in r, t[K] \neq \text{NULL} \quad (26)$$

Dalam SQL, **primary key** otomatis menggabungkan sifat unik dan non-null. Karena itu, atribut primary key tidak perlu diberi **not null** secara terpisah.

Entity integrity berhubungan langsung dengan referential integrity. Foreign key hanya bermakna jika relasi yang dirujuk memiliki key yang stabil, unik, dan non-null.

#### 12.2.5 Referential Integrity

##### Inti Konsep.

Referential integrity menjaga konsistensi hubungan antarrelasi dengan memastikan setiap foreign key merujuk tuple yang benar-benar ada, atau bernilai null jika diizinkan. Constraint ini penting karena relasi-relasi dalam database saling terhubung melalui key.

Motivasi referential integrity adalah mencegah **dangling reference**, yaitu nilai foreign key yang menunjuk data induk yang tidak ada. Contohnya, jika `course.dept_name` merujuk `department.dept_name`, maka setiap departemen pada `course` harus benar-benar ada di tabel `department`.

Secara formal, misalkan atribut  $A$  pada relasi  $R$  menjadi foreign key yang merujuk primary key  $A$  pada relasi  $S$ . Untuk setiap tuple  $t \in R$ , nilai  $t[A]$  harus memenuhi:

$$t[A] \in \Pi_A(S) \quad \text{atau} \quad t[A] = \text{NULL} \quad (27)$$

jika nilai null memang diizinkan.

Mekanisme pemeriksaan:

1. Saat tuple anak dimasukkan atau foreign key diperbarui, DBMS mencari nilai yang sama pada key relasi induk.
2. Jika nilai induk ditemukan, operasi diterima.

3. Jika tidak ditemukan dan foreign key boleh null, nilai null dapat diterima.
4. Jika tidak ditemukan dan nilai bukan null, operasi ditolak.
5. Saat tuple induk dihapus atau key induk diperbarui, DBMS menjalankan referential action yang didefinisikan.

Referential integrity adalah perekat struktur database relasional. Ia memakai konsep key dari relational model, dideklarasikan dengan DDL SQL, dan menjadi hasil konkret dari relationship pada E-R model.

### 12.2.6 SQL Constraints pada Relasi Tunggal

#### Inti Konsep.

SQL menyediakan constraint deklaratif seperti `not null`, `primary key`, `unique`, dan `check` untuk menjaga validitas tuple dalam satu relasi. Constraint ini penting karena DBMS dapat langsung menolak operasi DML yang melanggar skema.

Motivasi SQL constraints adalah memindahkan aturan data dari aplikasi ke struktur database. Aturan yang dideklarasikan di tabel berlaku konsisten untuk semua pengguna dan semua aplikasi.

| Constraint               | Definisi dan Contoh                                                                                                                                          |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>not null</code>    | Atribut tidak boleh bernilai null. Contoh: <code>name varchar(20) not null</code> .                                                                          |
| <code>primary key</code> | Himpunan atribut menjadi key utama, unik, dan non-null. Contoh: <code>primary key (ID)</code> .                                                              |
| <code>unique</code>      | Himpunan atribut harus unik seperti candidate key, tetapi SQL masih dapat mengizinkan null kecuali dilarang eksplisit. Contoh: <code>unique (email)</code> . |
| <code>check (P)</code>   | Nilai tuple harus memenuhi predikat <i>P</i> . Contoh: <code>check (semester in ('Fall', 'Winter', 'Spring', 'Summer'))</code> .                             |
| Named constraint         | Constraint diberi nama eksplisit agar mudah dirujuk atau dihapus. Contoh: <code>constraint minsalary check (salary &gt; 0)</code> .                          |

Tabel 42: Constraint SQL pada relasi tunggal

Contoh struktur:

Listing 27: Contoh constraint relasi tunggal

```

1 create table department (
2     dept_name varchar(20),
3     building varchar(15),
4     budget numeric(12,2) not null,
5     primary key (dept_name),
6     constraint positive_budget check (budget > 0)
7 );

```

`check` dapat mengekspresikan predikat sederhana. Secara teoretis, SQL standard mengizinkan subquery dalam `check`, tetapi sumber menekankan bahwa implementasi komersial umumnya tidak mendukung pemeriksaan subquery semacam itu dengan baik. Untuk constraint lintas tabel, foreign key, assertions, atau triggers lebih relevan.

### 12.2.7 Foreign Key dan Referential Actions

#### Inti Konsep.

Foreign key menghubungkan satu relasi dengan relasi lain melalui key yang dirujuk. Referential action menentukan apa yang dilakukan DBMS ketika tuple induk yang dirujuk dihapus atau key-nya diperbarui.

Motivasi referential action adalah operasi pada parent table dapat berdampak ke child table. Jika departemen dihapus, apa yang terjadi pada course yang merujuk departemen itu? Tanpa aturan eksplisit, database dapat menjadi tidak konsisten.

Sintaks dasar:

Listing 28: Contoh foreign key dengan referential action

```
1 create table course (
2     course_id varchar(8),
3     title varchar(50),
4     dept_name varchar(20),
5     credits numeric(2,0),
6     primary key (course_id),
7     foreign key (dept_name) references department
8         on delete set null
9         on update cascade
10 );
```

Target foreign key di SQL harus berupa primary key atau atribut yang dideklarasikan **unique**. Ini penting agar nilai foreign key menunjuk entitas induk secara tidak ambigu.

| Aksi               | Makna                                                                                                                                                      |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>restrict</b>    | Operasi pada parent diblokir jika masih ada tuple child yang merujuknya.                                                                                   |
| <b>no action</b>   | Operasi dievaluasi tanpa cascade bawaan; constraint tetap harus benar pada waktu pemeriksaan yang berlaku.                                                 |
| <b>cascade</b>     | Penghapusan atau perubahan key parent diteruskan otomatis ke tuple child yang merujuk.                                                                     |
| <b>set null</b>    | Foreign key pada child diset menjadi null ketika parent hilang atau berubah. Tidak boleh digunakan jika foreign key juga bagian primary key yang non-null. |
| <b>set default</b> | Foreign key pada child diset ke nilai default yang ditentukan.                                                                                             |

Tabel 43: Referential actions pada foreign key

Pada entitas lemah, foreign key biasanya merupakan bagian dari primary key. Karena primary key tidak boleh null, aksi **set null** tidak cocok untuk kasus tersebut.

### 12.2.8 Deferred Constraint Checking

#### Inti Konsep.

Deferred constraint checking adalah penundaan pemeriksaan constraint sampai transaksi selesai. Konsep ini penting karena beberapa transaksi legal dapat melanggar constraint sementara pada langkah awal, lalu memperbaikinya pada langkah akhir.

Motivasinya muncul pada **transitory violation**, yaitu pelanggaran sementara selama transaksi. Dalam transaksi multi-langkah, keadaan antara mungkin tidak konsisten, tetapi keadaan akhir konsisten. Jika DBMS memeriksa constraint setelah setiap statement secara ketat, transaksi semacam itu dapat gagal padahal hasil akhirnya valid.

Contoh sumber adalah relasi **person** yang memiliki foreign key **mother** dan **father** yang merujuk ke **person** sendiri. Ini menciptakan self-reference. Jika data orang tua dan anak saling bergantung dalam urutan penyisipan, sistem dapat mengalami kesulitan menemukan tuple yang dirujuk pada langkah pertama.

Solusi yang mungkin:

1. Sisipkan tuple orang tua lebih dahulu, lalu sisipkan anak.
2. Sisipkan anak dengan **mother** dan **father** bernilai null, lalu perbarui setelah data orang tua tersedia.
3. Gunakan deferred constraint checking agar pemeriksaan ditunda sampai transaksi selesai.

Pseudocode konseptual:

Listing 29: Pseudocode deferred constraint checking

```

1 begin transaction
2   mark selected constraints as deferred
3   execute insert/update/delete statements
4   at commit:
5     evaluate all deferred constraints
6     if every constraint holds then
7       commit
8     else
9       rollback
10 end transaction

```

Konsep ini berkaitan dengan transaction control. Constraint tetap wajib benar, tetapi waktu pemeriksaannya dapat digeser dari statement-level ke transaction-level.

## 12.2.9 Assertions, Triggers, Functions, dan Procedures

### Inti Konsep.

Assertions, triggers, functions, dan procedures digunakan ketika constraint sederhana tidak cukup mengekspresikan aturan bisnis kompleks. Mereka penting karena aturan lintas tabel atau aturan prosedural sering tidak dapat ditulis hanya dengan **primary key**, **foreign key**, atau **check**.

Motivasi fitur lanjutan ini adalah batasan SQL DDL dasar. Banyak aturan bisnis membutuhkan logika lintas relasi, agregasi, atau reaksi otomatis terhadap event tertentu.

| Fitur            | Definisi dan Peran                                                                                                                                                                                             |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assertion</b> | Predikat deklaratif yang wajib selalu bernilai benar, ditulis secara konseptual sebagai <b>create assertion name check (predicate)</b> . Cocok untuk constraint global, tetapi mahal dan jarang didukung luas. |
| <b>Trigger</b>   | Statement yang dijalankan otomatis sebagai efek samping event seperti <b>insert</b> , <b>delete</b> , atau <b>update</b> . Trigger memiliki kondisi pemicu dan aksi.                                           |
| <b>Function</b>  | Modul yang mengembalikan nilai dan dapat dipakai untuk logika bisnis yang lebih kompleks.                                                                                                                      |
| <b>Procedure</b> | Modul prosedural yang menjalankan rangkaian aksi di database, sering dipakai untuk aturan bisnis atau proses operasional.                                                                                      |

Tabel 44: Fitur lanjutan untuk constraint dan aturan bisnis

Contoh trigger konseptual:

Listing 30: Contoh struktur trigger konseptual

```

1 create trigger normalize_grade
2 after update of grade on takes
3 referencing old row as old_tuple new row as new_tuple
4 for each row
5 when (new_tuple.grade = '')
6 begin
7     update takes
8     set grade = null
9     where ID = new_tuple.ID;
10 end;
```

Assertion bersifat deklaratif, sedangkan trigger bersifat reaktif dan prosedural. Sumber menekankan bahwa assertion sering mahal karena DBMS harus memastikan predikat global selalu benar. Dalam praktik, trigger atau application logic kadang dipakai, tetapi semakin prosedural aturannya, semakin besar risiko kompleksitas dan efek samping.

### 12.2.10 Hubungan Integrity Constraints dengan Desain Relasional

#### Inti Konsep.

Integrity constraints adalah bentuk implementasi dari key, dependency, dan hubungan antarrelasi yang ditemukan pada desain relasional. Mereka penting karena normalisasi menghasilkan struktur tabel, tetapi constraint memastikan struktur itu tetap konsisten saat digunakan.

Hubungan pertama adalah dengan **keys**. Entity integrity bergantung pada primary key; referential integrity bergantung pada foreign key yang merujuk primary key atau unique key. Tanpa key yang benar, constraint antarrelasi tidak dapat didefinisikan dengan aman.

Hubungan kedua adalah dengan **DDL SQL**. Constraint dideklarasikan dalam **create table** atau **alter table**. Setelah constraint menjadi bagian dari skema, semua operasi DML harus mematuhi.

Hubungan ketiga adalah dengan **Relational Database Design**. Normalisasi berdasarkan FD mengurangi anomali, tetapi beberapa FD kompleks tidak mudah ditegakkan oleh SQL kecuali melalui primary key, unique, assertion, atau trigger. Sumber menekankan bahwa assertion dapat mahal, sehingga desain praktis sering mempertimbangkan trade-off antara BCNF, dependency preservation, dan biaya penegakan constraint.

## 12.3 Algoritma dan Rumus Penting

### 12.3.1 Validasi Penambahan Constraint

#### Tujuan Algoritma.

Algoritma ini menggambarkan cara DBMS menerima constraint baru pada database yang sudah berisi data. Constraint hanya boleh diterima jika semua data saat ini sudah mematuhi.

Listing 31: Pseudocode validasi constraint baru

```

1 Input: database instance D, new constraint C
2 Output: accept or reject
3
4 for each relevant tuple or relation state in D do
5     if C is violated then
6         reject C
7         stop
8 add C to schema
9 enforce C for all future modifications
```

### 12.3.2 Kondisi Referential Integrity

#### Makna Rumus.

Rumus referential integrity menyatakan bahwa nilai foreign key pada relasi anak harus muncul sebagai key pada relasi induk, kecuali jika nilai null diizinkan.

$$\forall t \in R, t[A] \in \Pi_A(S) \vee t[A] = \text{NULL} \quad (28)$$

dengan:

- $R$ : relasi anak yang memiliki foreign key.
- $S$ : relasi induk yang dirujuk.
- $A$ : atribut foreign key di  $R$  sekaligus key yang dirujuk di  $S$ .
- $\Pi_A(S)$ : himpunan nilai atribut  $A$  yang ada pada relasi  $S$ .

### 12.3.3 Pemeriksaan Operasi DML terhadap Constraint

#### Tujuan Algoritma.

Pseudocode ini merangkum cara DBMS mengevaluasi operasi modifikasi terhadap constraint sebelum perubahan dibuat permanen.

Listing 32: Pseudocode pemeriksaan DML terhadap constraint

```
1 Input: operation op, database D, constraints C
2 Output: commit operation or reject operation
3
4 tentatively apply op to produce D_new
5 for each constraint c in C do
6     if c is immediate and D_new violates c then
7         reject op
8         restore D
9         stop
10 if transaction commits then
11     for each deferred constraint c in C do
12         if final transaction state violates c then
13             rollback transaction
14             stop
15 commit changes
```

## 13 Model Data Multidimensi & Data Warehouse

Model Data Multidimensi dan Data Warehouse membahas cara menyusun data untuk analisis historis dan pengambilan keputusan. Jika database operasional dirancang agar transaksi harian cepat dan konsisten, data warehouse dirancang agar pertanyaan analitik seperti tren penjualan, performa cabang, dan perilaku pelanggan dapat dijawab dengan query agregasi yang lebih sederhana.

Materi ini menunjukkan bahwa desain basis data tidak selalu mengejar normalisasi maksimum. Untuk beban kerja analitik yang dominan baca dan agregasi, struktur seperti star schema sering sengaja menyimpan redundansi terkontrol agar query lebih efisien.

### Komentar 13.1: Keterbatasan Sumber

Query mendalam NotebookLM untuk materi ini beberapa kali gagal di tengah jawaban. Penyusunan bagian ini tetap memakai outline final yang sudah diperoleh dari NotebookLM pada workflow pra-penulisan, ditambah pengetahuan umum yang ditandai dalam environment cmt.

### 13.1 Outline Fundamental Konsep Materi

1. **WAJIB** Latar belakang decision support
  - (a) **WAJIB** Kebutuhan analisis eksekutif
  - (b) **WAJIB** Masalah query analitik pada database operasional
2. **WAJIB** OLTP dan OLAP
  - (a) **WAJIB** Perbedaan tujuan, pengguna, desain, data, dan pola akses
  - (b) **WAJIB** Alasan beban OLAP dipisah dari OLTP
3. **WAJIB** Data warehouse
  - (a) **WAJIB** Repositori historis terpadu
  - (b) **WAJIB** Unified schema
  - (c) **PAHAM** ETL/ELT dan data loaders
4. **WAJIB** Model data multidimensi
  - (a) **WAJIB** Fact, measure, dimension, member
  - (b) **WAJIB** Data cube dan hypercube
  - (c) **PAHAM** Grain fact table
5. **WAJIB** Warehouse schema
  - (a) **WAJIB** Fact table dan dimension table
  - (b) **WAJIB** Star schema
  - (c) **WAJIB** Snowflake schema
  - (d) **PAHAM** Fact constellation atau galaxy schema
6. **WAJIB** Query dan operasi OLAP
  - (a) **WAJIB** Simplifikasi query analitik
  - (b) **WAJIB** Roll-up dan drill-down
  - (c) **PAHAM** Slice, dice, pivot, cross-tabulation
  - (d) **PAHAM** cube dan rollup

7. **PAHAM** Hubungan dengan materi lain
- (a) **PAHAM** Hubungan dengan E-R model
  - (b) **PAHAM** Hubungan dengan normalisasi
  - (c) **PAHAM** Additive, semi-additive, dan non-additive measures

## 13.2 Penjelasan Konsep

### 13.2.1 Latar Belakang Decision Support

#### Inti Konsep.

Decision support membutuhkan data historis yang siap dianalisis untuk membantu manajemen mengambil keputusan. Konsep ini penting karena pertanyaan analitik berbeda dari transaksi harian: ia jarang dieksekusi, tetapi berat, agregatif, dan sering menyentuh banyak data.

Motivasi data warehouse berawal dari kebutuhan eksekutif untuk memahami tren dan faktor bisnis. Contoh pertanyaan dari sumber meliputi total penjualan barang setiap bulan, rata-rata rupiah penjualan per bulan, penjual terbaik, kota target pasar terbesar, dan barang yang paling laku.

Database operasional relasional biasanya sangat baik untuk transaksi kecil seperti mencatat order, memperbarui stok, atau menyisipkan pembayaran. Namun pertanyaan analitik sering membutuhkan agregasi historis dan banyak join. Contoh dari sumber: untuk mencari penjual top kategori *skin care* pada tahun tertentu, query relasional perlu menjumlahkan data order melalui banyak tabel seperti `order`, `order_item`, `product`, `category`, dan `seller`.

Mekanisme masalahnya:

1. Data operasional disimpan terpisah mengikuti desain aplikasi dan normalisasi.
2. Query analitik perlu menggabungkan banyak tabel untuk memperoleh konteks bisnis.
3. Query melakukan agregasi besar seperti `SUM`, `AVG`, dan `GROUP BY`.
4. Eksekusi berat ini dapat mengganggu transaksi harian jika dijalankan langsung pada database OLTP.

Konsep ini menjadi alasan utama pemisahan OLTP dan OLAP. Data warehouse bukan sekadar salinan database, tetapi struktur baru yang disusun agar pertanyaan analitik menjadi lebih langsung.

### 13.2.2 OLTP dan OLAP

#### Inti Konsep.

OLTP menangani transaksi operasional harian, sedangkan OLAP menangani analisis historis untuk keputusan. Perbedaan ini penting karena desain schema, pola akses, dan optimasi keduanya berlawanan.

OLTP (*Online Transaction Processing*) dipakai untuk operasi bisnis sehari-hari. Karakteristiknya adalah transaksi sering, kecil, dan membutuhkan konsistensi update yang kuat. OLAP (*Online Analytical Processing*) dipakai untuk membaca dan menganalisis data besar, historis, dan teragregasi.



| Aspek        | OLTP                                                                 | OLAP                                                                |
|--------------|----------------------------------------------------------------------|---------------------------------------------------------------------|
| Pengguna     | Clerk, operator, aplikasi transaksi, IT professional.                | Knowledge worker, analis, manajemen, eksekutif.                     |
| Fungsi       | Operasi harian: insert, update, delete, lookup kecil.                | Decision support: analisis tren, agregasi, pelaporan.               |
| Desain skema | Application-oriented, sering berasal dari E-R model dan normalisasi. | Subject-oriented, sering memakai star schema atau snowflake schema. |
| Isi data     | Data current, detail, dan terisolasi per aplikasi.                   | Data historis, consolidated, dan terintegrasi.                      |
| Pola akses   | Banyak update interaktif dan transaksi pendek.                       | Read-mostly, query kompleks, agregasi besar.                        |

Tabel 45: Perbandingan OLTP dan OLAP

Alasan pemisahannya adalah beban kerja. Query OLAP yang berat dapat memakai banyak CPU, I/O, dan memori. Jika query seperti itu berjalan di sistem OLTP yang sedang melayani transaksi pelanggan, performa operasional dapat turun. Data warehouse memindahkan beban analitik ke sistem yang memang dirancang untuk itu.

### 13.2.3 Data Warehouse

#### Inti Konsep.

Data warehouse adalah repositori informasi historis yang dikumpulkan dari berbagai sumber dan disimpan dalam satu skema terpadu. Ia digunakan untuk menyederhanakan query analitik dan memindahkan beban decision support dari sistem transaksi operasional.

Motivasi data warehouse adalah data perusahaan sering tersebar di banyak sistem: penjualan, inventori, pembayaran, pengiriman, dan customer service. Setiap sistem dapat memiliki skema dan istilah berbeda. Untuk analisis manajerial, data tersebut perlu disatukan agar dapat dibaca sebagai satu pandangan bisnis.

**Unified schema** adalah skema terpadu yang menyelaraskan data dari berbagai sumber. Misalnya, satu sistem menyebut pelanggan sebagai **customer**, sistem lain menyebutnya **buyer**; warehouse perlu menyatukan definisi ini agar laporan tidak ambigu.

Mekanisme umum pembangunan warehouse:

1. Ambil data dari sumber operasional.
2. Bersihkan dan standarkan data.
3. Gabungkan data ke skema terpadu.
4. Simpan data historis pada struktur warehouse.
5. Jalankan query OLAP dan reporting di atas warehouse, bukan langsung di OLTP.

#### Komentar 13.2: Catatan: Sumber Pengetahuan Umum

Proses pemindahan data ke warehouse umum disebut ETL (*Extract, Transform, Load*) atau ELT (*Extract, Load, Transform*). ETL mengekstrak data dari sumber, mentransformasikannya lebih dulu, lalu memuatnya ke warehouse. ELT memuat data mentah ke platform tujuan lebih awal, kemudian transformasi dilakukan di dalam platform analitik. Outline NotebookLM menyebut data loaders dan contoh alat seperti Apache NiFi, Talend, Kafka, Teradata, SAP HANA, dan Amazon Redshift.

Data warehouse berhubungan dengan integrity dan relational design, tetapi tujuannya berbeda. Sistem OLTP mengutamakan update konsisten; warehouse mengutamakan pembacaan historis dan agregasi cepat.

#### 13.2.4 Model Data Multidimensi

##### Inti Konsep.

Model data multidimensi menyusun data analitik sebagai fakta yang diukur dari beberapa sudut pandang atau dimensi. Model ini penting karena query bisnis biasanya bertanya tentang ukuran numerik, seperti penjualan, berdasarkan waktu, produk, lokasi, atau penjual.

Motivasi model multidimensi adalah pertanyaan analitik hampir selalu memiliki bentuk: “berapa ukuran bisnis tertentu jika dilihat menurut dimensi tertentu?” Contoh: total penjualan menurut bulan, produk, dan kota.

| Istilah          | Definisi atau Peran                                                                                                                  |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fact</b>      | Kejadian bisnis yang dianalisis, misalnya satu transaksi penjualan atau pengiriman barang.                                           |
| <b>Measure</b>   | Nilai kuantitatif yang diukur dari fact, misalnya <b>quantity</b> , <b>receipts</b> , <b>units_sold</b> , atau <b>dollars_sold</b> . |
| <b>Dimension</b> | Sudut pandang analisis terhadap measure, misalnya waktu, produk, toko, lokasi, penjual, atau pembeli.                                |
| <b>Member</b>    | Nilai aktual dari dimensi, misalnya produk <i>Shiny</i> , toko <i>EverMore</i> , atau tanggal tertentu.                              |
| <b>Data cube</b> | Metafora struktur multidimensi; sel kubus menyimpan measure, tepi kubus merepresentasikan dimensi.                                   |
| <b>Hypercube</b> | Data cube dengan lebih dari tiga dimensi.                                                                                            |

Tabel 46: Komponen model data multidimensi

Contoh sumber memakai sales data cube dengan dimensi **store**, **product**, dan **date**. Satu sel dapat merepresentasikan penjualan produk *Shiny* di toko *EverMore* pada tanggal tertentu, dengan measure seperti **quantity** = 10 dan **receipts** = 25.

##### Komentar 13.3: Catatan: Sumber Pengetahuan Umum

**Grain** fact table adalah tingkat detail paling rendah yang direpresentasikan oleh satu baris fact. Misalnya, grain dapat berupa “satu baris per item terjual per transaksi”, atau “satu baris per produk per toko per hari”. Grain harus ditentukan jelas sebelum memilih dimensi dan measure, karena grain menentukan makna setiap baris fact.

Model multidimensi berkaitan dengan SQL karena hasil akhirnya tetap sering diimplementasikan sebagai tabel relasional. Perbedaannya ada pada orientasi desain: tabel disusun agar agregasi bisnis mudah, bukan agar semua redundansi hilang.

#### 13.2.5 Fact Table dan Dimension Table

##### Inti Konsep.

Fact table menyimpan kejadian dan measure utama, sedangkan dimension table menyimpan konteks deskriptif untuk menganalisis measure tersebut. Pemisahan ini penting karena query analitik biasanya mengagregasi measure berdasarkan atribut dimensi.

**Fact table** biasanya besar karena memuat banyak kejadian bisnis. Atributnya terdiri dari **measure attributes** dan **dimension attributes**. Measure attributes adalah angka yang dianalisis; dimension attributes biasanya foreign key ke tabel dimensi.

**Dimension table** biasanya lebih kecil dan menyimpan informasi deskriptif. Contoh dimensi produk dapat menyimpan nama produk, kategori, merek, atau supplier. Data yang tidak memberi makna analitik tidak perlu selalu disimpan di dimensi.

| Komponen             | Peran                                                                                 |
|----------------------|---------------------------------------------------------------------------------------|
| Fact table           | Menyimpan foreign key ke dimensi dan measure yang dihitung atau diagregasi.           |
| Dimension table      | Menyimpan deskripsi konteks untuk memfilter, mengelompokkan, atau melabeli fact.      |
| Measure attributes   | Nilai numerik seperti jumlah unit, nilai penjualan, dan rata-rata penjualan.          |
| Dimension attributes | Kunci atau atribut pengelompokan seperti waktu, lokasi, barang, penjual, dan pembeli. |

Tabel 47: Fact table dan dimension table

Pada contoh e-commerce dari sumber, fact penjualan dapat memuat measure seperti `Jumlah_penjualan_barang` dan `Jumlah_rupiah_penjualan`, lalu dikelilingi dimensi seperti `Waktu`, `Barang`, `Penjual`, `Pembeli`, dan bahkan sumber eksternal seperti `Cuaca`.

### 13.2.6 Star, Snowflake, dan Galaxy Schema

#### Inti Konsep.

Star, snowflake, dan galaxy schema adalah pola struktur data warehouse. Star schema menempatkan satu fact table di tengah dikelilingi dimensi; snowflake menormalisasi sebagian dimensi; galaxy schema memakai beberapa fact table yang berbagi dimensi.

Motivasi pola schema adalah menyusun fact dan dimension agar query analitik mudah dibaca dan efisien. Setiap pola memberi trade-off antara kesederhanaan query, redundansi, dan struktur hierarki dimensi.

| Schema                             | Struktur                                                  | Implikasi                                                                                                      |
|------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Star schema                        | Satu fact table pusat dikelilingi dimension tables.       | Query sederhana dan cepat dibaca; dimensi cenderung denormalized.                                              |
| Snowflake schema                   | Dimensi dipecah menjadi tabel hierarkis yang lebih kecil. | Redundansi dimensi berkurang, tetapi query butuh lebih banyak join.                                            |
| Fact constellation / Galaxy schema | Beberapa fact table berbagi beberapa dimension table.     | Cocok untuk proses bisnis lebih kompleks, misalnya sales dan shipping berbagi dimensi waktu, item, dan lokasi. |

Tabel 48: Pola warehouse schema

Sumber memberi contoh star schema dengan Sales Fact Table yang memiliki measure seperti `units_sold`, `dollars_sold`, dan `avg_sales`, terhubung ke dimensi `time`, `branch`, `item`, dan `location`. Pada snowflake schema, dimensi seperti item dapat dipecah ke supplier, dan location dapat dipecah ke city. Pada galaxy schema, Shipping Fact Table dapat ditambahkan dan berbagi dimensi `time`, `item`, dan `location`.

Hubungannya dengan normalisasi bersifat pragmatis. Star schema sengaja menoleransi redundansi dimensi agar query lebih sederhana; snowflake schema menerapkan normalisasi pada dimensi tertentu ketika struktur hierarkis perlu dirapikan.

### 13.2.7 Query dan Operasi OLAP

#### Inti Konsep.

Operasi OLAP adalah cara menavigasi data multidimensi melalui agregasi, perincian, pemilihan irisan data, dan perubahan perspektif tampilan. Operasi ini penting karena pengguna analitik perlu bergerak cepat dari ringkasan besar ke detail tertentu.

Motivasi operasi OLAP adalah laporan bisnis jarang berhenti pada satu query. Setelah melihat total penjualan nasional, analis mungkin ingin turun ke provinsi, lalu kota, lalu toko. Setelah melihat satu bulan, analis dapat membandingkan kuartal atau tahun.

| Operasi                     | Makna                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------|
| Roll-up                     | Menaikkan level agregasi, misalnya dari hari ke bulan atau dari kota ke provinsi.           |
| Drill-down                  | Menurunkan level agregasi ke detail lebih halus, misalnya dari tahun ke kuartal ke bulan.   |
| Slice                       | Memilih satu nilai dimensi, misalnya hanya tahun 2026.                                      |
| Dice                        | Memilih sub-kubus dengan beberapa rentang atau nilai dimensi.                               |
| Pivot /<br>Cross-tabulation | Mengubah orientasi tampilan dimensi, misalnya produk sebagai baris dan bulan sebagai kolom. |
| cube dan rollup             | Ekstensi agregasi SQL untuk menghitung banyak kombinasi grouping.                           |

Tabel 49: Operasi OLAP

#### Komentar 13.4: Catatan: Sumber Pengetahuan Umum

Ukuran atau *measure* sering diklasifikasikan menjadi additive, semi-additive, dan non-additive. Measure additive dapat dijumlahkan pada semua dimensi, misalnya `sales_amount`. Measure semi-additive dapat dijumlahkan pada sebagian dimensi tetapi tidak semua, misalnya saldo akun yang dapat dijumlahkan antar akun tetapi tidak tepat dijumlahkan sepanjang waktu. Measure non-additive tidak boleh dijumlahkan langsung, misalnya rasio atau persentase; biasanya perlu dihitung ulang dari komponen dasarnya.

Operasi OLAP berkaitan dengan SQL karena agregasi akhirnya dapat diekspresikan dengan `GROUP BY`, `ROLLUP`, `CUBE`, window function, atau query khusus tools BI. Model multidimensi membuat query ini lebih pendek karena banyak join operasional sudah disederhanakan ke fact-dimension schema.

### 13.2.8 Hubungan dengan E-R Model dan Normalisasi

#### Inti Konsep.

Data warehouse berangkat dari data operasional yang sering dirancang dengan E-R model dan normalisasi, tetapi mengubah orientasinya menjadi subject-oriented dan analitik. Hubungan ini penting karena desain OLTP yang baik belum tentu menjadi desain OLAP yang baik.

Pada OLTP, E-R model membantu menangkap entitas dan hubungan aplikasi secara detail. Hasilnya sering dinormalisasi untuk mengurangi redundansi dan menjaga update konsisten. Pada OLAP, data direstrukturisasi menjadi fact dan dimension agar pembacaan historis cepat.

Hubungan dengan normalisasi berupa trade-off:

1. Normalisasi OLTP mengurangi redundansi dan anomali update.
2. Warehouse sering read-mostly sehingga biaya update bukan fokus utama.
3. Star schema menoleransi redundansi dimensi agar query sederhana.
4. Snowflake schema menerapkan sebagian normalisasi pada dimensi ketika hierarki perlu dipisah.

Materi ini menutup rangkaian desain basis data dengan menunjukkan bahwa kualitas desain selalu bergantung pada beban kerja. Untuk transaksi, desain yang baik berarti konsisten dan minim redundansi. Untuk analitik, desain yang baik berarti historis, terpadu, dan mudah diagregasi.

### 13.3 Algoritma dan Rumus Penting

#### 13.3.1 Alur Pembangunan Data Warehouse

##### Tujuan Algoritma.

Alur ini merangkum proses memindahkan data dari sumber operasional ke warehouse agar siap dipakai untuk query analitik.

Listing 33: Pseudocode alur pembangunan data warehouse

```
1 Input: operational data sources
2 Output: analytical warehouse
3
4 for each source system do
5     extract relevant current and historical data
6     clean invalid or inconsistent values
7     transform names, formats, and meanings into unified schema
8     load transformed data into warehouse tables
9 build fact tables and dimension tables
10 run OLAP queries and reporting on warehouse
```

#### 13.3.2 Bentuk Umum Query Agregasi OLAP

##### Makna Rumus.

Bentuk ini menunjukkan pola umum query OLAP: measure dihitung dari fact table, lalu dikelompokkan menurut atribut dimensi.

$$Result = \gamma_{D_1, D_2, \dots, D_k; agg(M)}(Fact \bowtie Dim_1 \bowtie \dots \bowtie Dim_k) \quad (29)$$

dengan:

- *Fact*: fact table.
- *Dim<sub>i</sub>*: dimension table ke-*i*.
- *D<sub>i</sub>*: atribut dimensi untuk grouping.
- *M*: measure yang dihitung.
- *agg*: fungsi agregasi seperti SUM, AVG, COUNT, MIN, atau MAX.